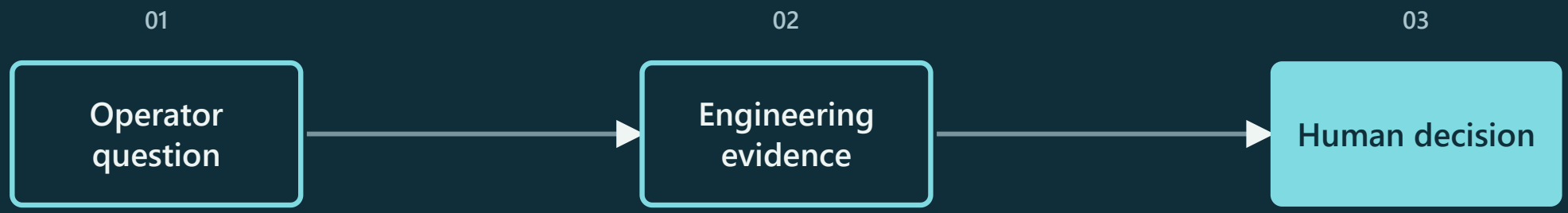
FOUNDATIONS

Agentic AI for grid operators

Agentic AI for grid operators · three-hour workshop



Understand the workflow

Run the engineering tools

Challenge the recommendation

> **Build an evidence trail before asking an operator to act.**

PowerAgent roadmap · IEEE PEM 2025

Three decisions; one discipline

Start with the operator

What must an operator see before accepting any of these?

The exact candidate, declared model, checked limits, evidence age, and unresolved assumptions.

Voltage: choose a continuous inverter command

Restoration: choose permitted paths and served loads

Cyber response: decide which measurements support control

> **The mathematical task changes; the requirement for traceable evidence remains.**

Tutorial notebooks · three declared case scopes

What you will be able to explain

Learning outcomes

01 Trace state through every proposed stage

02 Match code, equations, outputs, and timing boundaries

03 Separate numerical benefit from workflow benefit

If the final objective ties the baseline, what could still improve?

Inspectability or stale-evidence rejection may improve; those benefits need their own measurements.

> **Explain the graph, reproduce a decision, and defend an acceptance or hold.**

A three-hour route with experiments throughout

Predict → run → explain

Block	Work at the keyboard
Foundations	G0–G5; L0–L1
Voltage	L2–L5; archived and optional live Llama
Restoration / cyber	L6–L9
Evidence / operator	L10–L12

0–40 min: foundations and graph lab

40–82 min: voltage; 82–92 min: break

92–149 min: restoration and cyber response

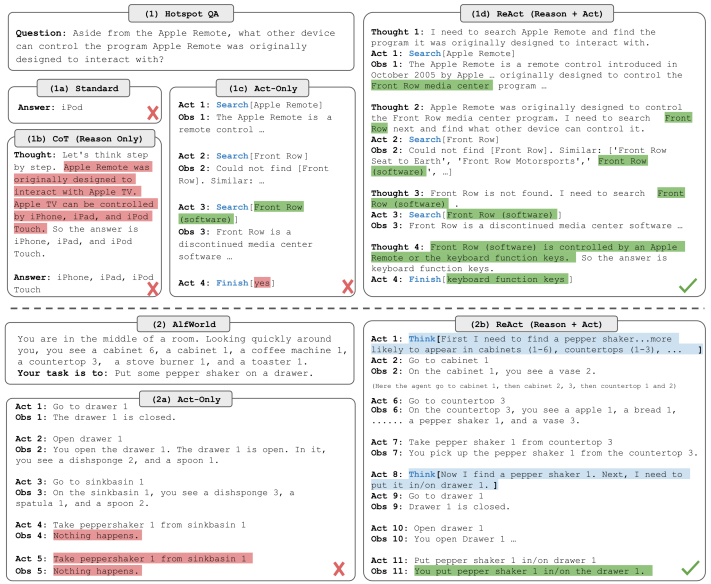
149–180 min: evaluation and operator capstone

> **Each case follows concepts, an executable experiment, and an evidence-based discussion.**

Graph laboratory Engineering laboratory

A useful agent closes a feedback loop

From answering to acting through tools



Yao et al. (ICLR 2023), ReAct Fig. 1, CC BY 4.0. Unchanged figure.

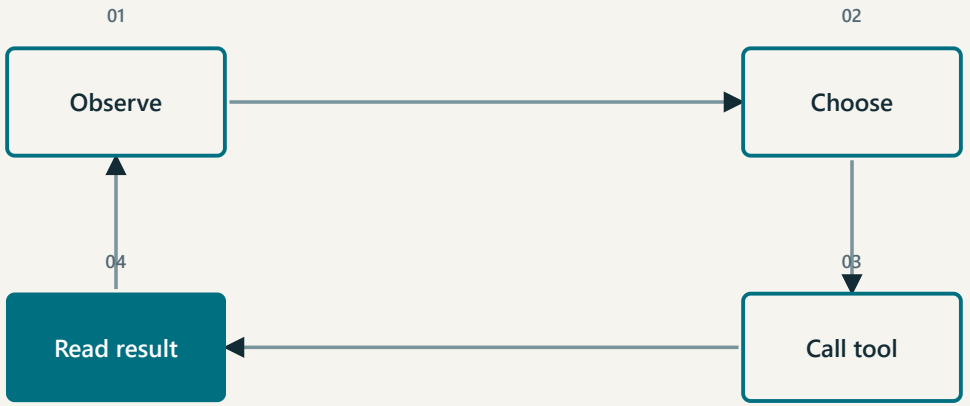
- Observe the current evidence
- Select an allowed operation
- Read its result before continuing

> **A returned tool result can change the next permitted operation.**

Yao et al. · ReAct · ICLR 2023 LangGraph · workflows and agents Figure and reuse license

Tool feedback changes the next choice

Teaching adaptation of the ReAct interaction pattern



A choice requests a bounded tool.

The tool returns external evidence.

New evidence can change the next choice.

> **The useful loop is a decision updated by an observation.**

Conceptual adaptation: Yao et al., ReAct (ICLR 2023), CC BY 4.0

Tool, agent, orchestrator: three different jobs

Name the responsibility

01

Agent chooses

02

Tool computes

03

Orchestrator coordinates

Agent: chooses an allowed next operation

Tool: returns a specific calculation or record

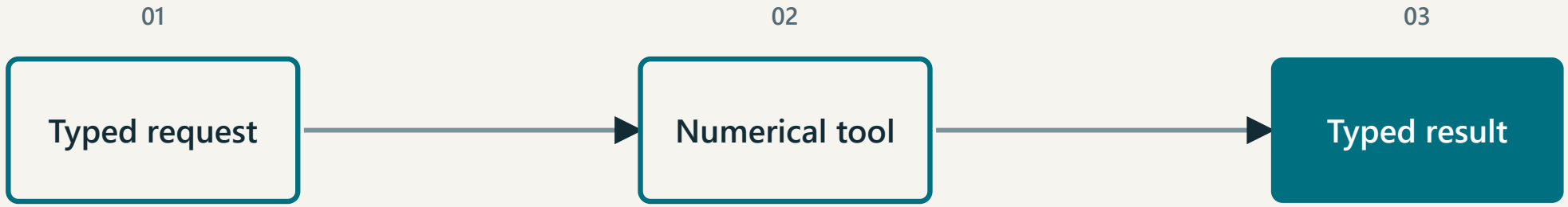
Orchestrator: runs routes and stores state

> **Roles describe responsibilities; they do not require different language-model backbones.**

Wu et al. · AutoGen · 2023 preprint Tutorial implementation

A tool call is a small engineering contract

Make the interface inspectable



Input: case, revision, command, and limits

Output: values, units, convergence, and evidence identity

Failure: explicit error or rejected result

> **A result needs a candidate, units, model scope, and status.**

MCP tools specification · 2025-11-25 Classroom evidence functions

Where MCP fits in the picture

Interoperability is a layer



Discover available tools

Submit structured arguments

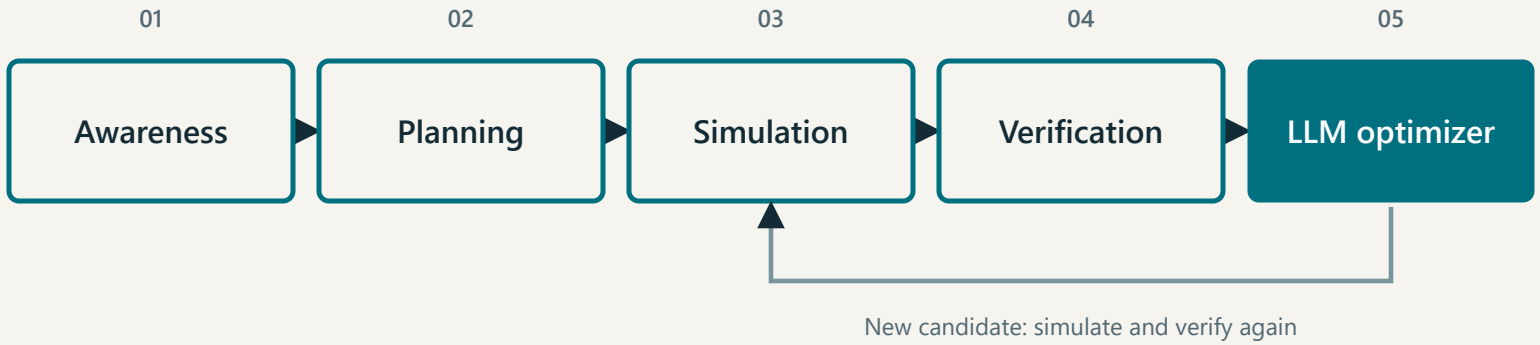
Receive a structured result

> **A common tool interface helps integration; it does not certify the returned engineering result.**

MCP tools specification · 2025-11-25 PowerMCP · official repository

The complete proposed workflow has five stages

One candidate, one connected record



Awareness establishes state and request

Planning proposes; simulation computes consequences

Verification checks; optimization chooses the next trial

Operator review receives supported evidence

> **Every changed candidate must return to simulation and verification.**

Executed five-stage tutorial implementation Five-stage archived evidence

Give each stage the method it needs

Language plus engineering methods

Responsibility	Returned artifact
Llama interpretation	Allowed operation + public explanation
Numerical method	Candidate + calculated values
Verifier	Pass/hold/reject + margins

Planning: MPC, restoration MILP, or estimation/control

Simulation: LinDistFlow, OpenDSS, or declared teaching model

Verification: explicit numerical and consistency checks

Selection: fixed rule, vote, or learned bandit

> **The language model coordinates bounded choices; numerical methods produce and check the physical evidence.**

Classroom methods Proposal case tools

Learn four symbols before reading the graph

A small visual vocabulary

A

An operation or node

Box A: an operation

s

The current record

Record s: current shared state

$A \rightarrow B$

B requires the result from A

Arrow A → B: B needs A's result

$r(s)$

A routing rule evaluated on state

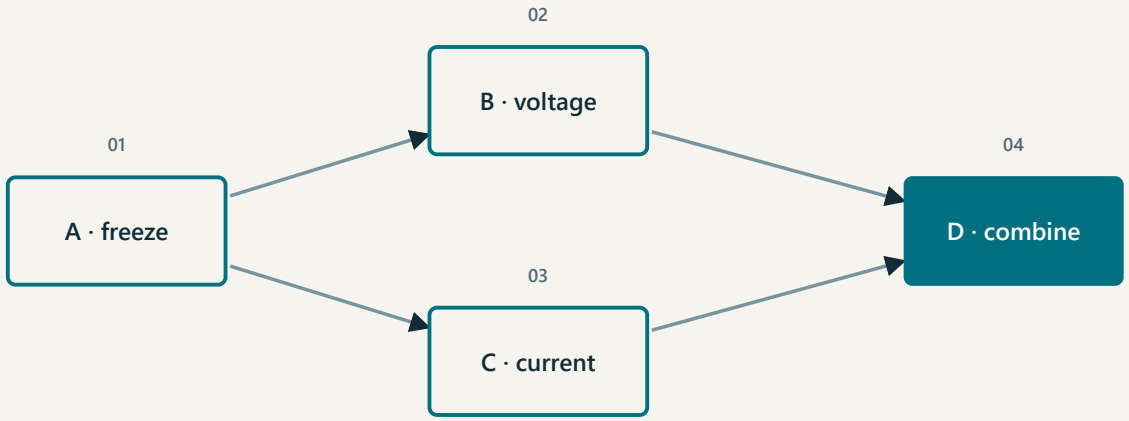
Diamond r(s): choose the permitted next route

> **Workflow arrows represent dependencies, not electrical connections.**

LangGraph graph API

DAG means directed and acyclic

Prerequisites first



Dependencies determine which tools may run together

$$G = (N, E), \qquad N = \{A, B, C, D\}$$

G Dependency graph N Set of operations E Set of directed prerequisite edges

A freezes the input

B and C inspect independent results

D combines both pieces of evidence

> **A DAG encodes dependencies without a backward path.**

NetworkX · topological sorting

Which schedule respects every arrow?

Predict before revealing

Which two orders are valid?

The first two. The third starts D before C has supplied its result.

A, B, C, D

A, C, B, D

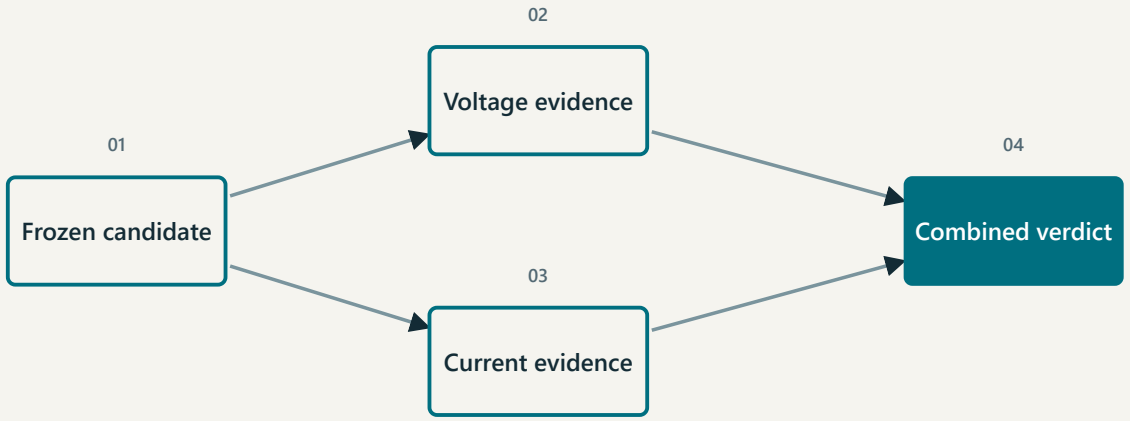
A, B, D, C

> **The join must follow both prerequisites, whatever their order.**

NetworkX · topological sorting

Fan-out and join need a shared identity

Parallel work is still one experiment



Dependencies determine which tools may run together

Fan-out copies a consistent input

Each branch returns its own evidence

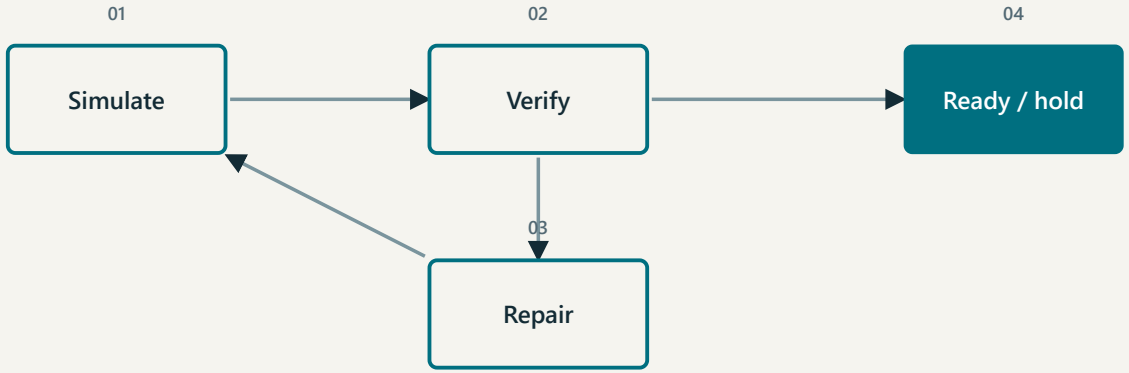
Join checks completeness and identity

> **Both branches must refer to the same frozen candidate and context.**

LangGraph graph construction examples Tutorial evidence binding

A repair loop is a cycle, with a budget

Bounded retries



Bounded repair budget • exhausted budget routes to HOLD

$$\text{repair only if } k < K$$

k Completed repairs K Maximum permitted repairs

Pass → ready for review

Fail with budget → repair and simulate again

Fail without budget → hold

> **A finite retry budget bounds attempts; it does not guarantee a feasible command.**

Executed retry policy · G2–G4

State is the laboratory record carried forward

Keep facts together

$$s_{t+1} = \mathrm{merge}(s_t, \Delta s_t)$$

s_t Record before workflow step t

Δs_t Returned field updates; not arithmetic subtraction

Candidate: command and context

Evidence: current, limit, and verdict

Control flow: attempts, budget, and status

Trace: the sequence of recorded events

> **A node reads state and returns explicit updates to the record.**

G1 state schema Python TypedDict

Replace a result; append an event

Reducers in one example

Should a repaired command keep the previous current as its current result?

$$R_{\text{trace}}(a, b) = a + b$$

R Reducer combining stored value and update a, b Existing event list and new event list

No. Replace the current result; preserve the previous event in the trace.

Current: 1.00 then 0.80 → keep 0.80

Trace: [simulate] plus [verify] → keep both

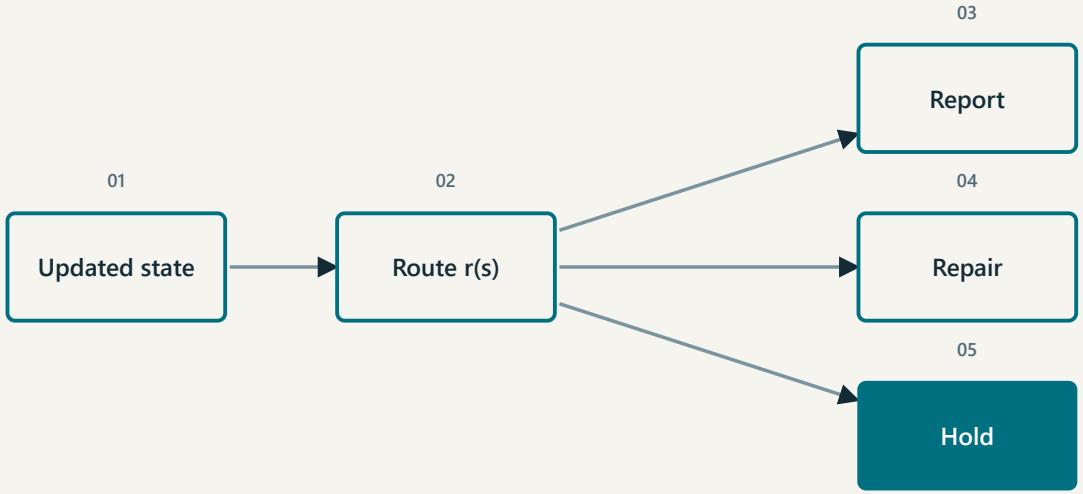
Parallel writes need an explicit compatible rule

> **The merge rule is part of the data model.**

G1 · list-addition reducer LangGraph · reducers

Routing turns evidence into the next step

Three explicit branches



passed = true → report

passed = false and k < K → repair

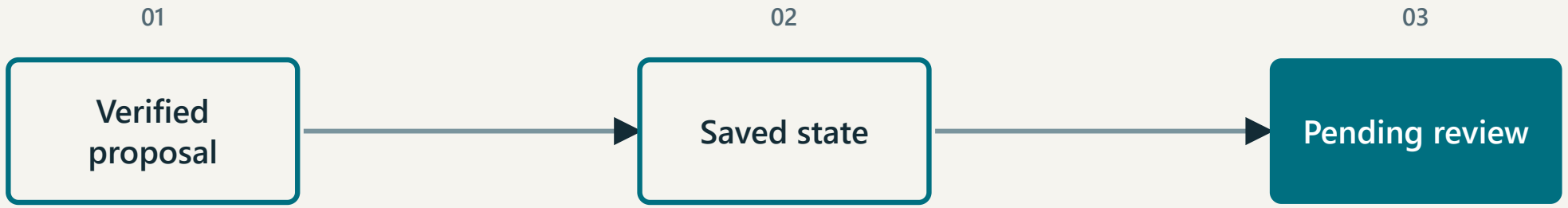
passed = false and k ≥ K → hold

> **Verification computes the verdict; routing chooses what to do with it.**

Exact route function · G2

Checkpoint: remember where the workflow stopped

Persistence



Save the current record

Keep the next node

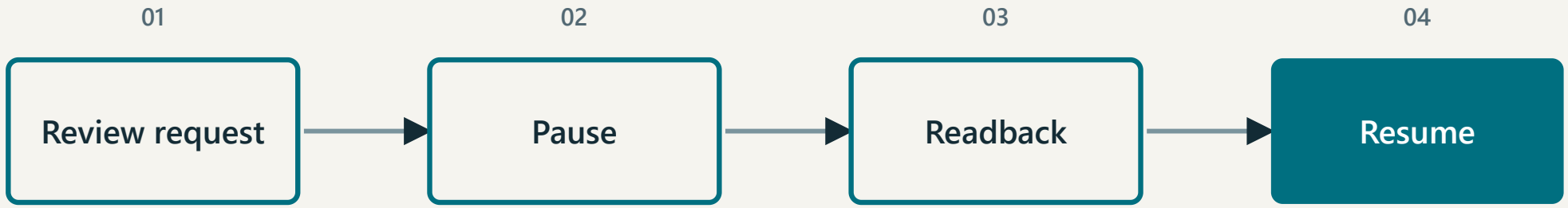
Resume the same execution context

> **A checkpoint associates execution state with a thread identifier.**

LangGraph · persistence G5 · InMemorySaver example

Interrupt: make the review decision visible

Human interaction



Pause with candidate and evidence scope

Inspect before responding

Resume with an explicit matching response

> **A review response must identify the recommendation it acknowledges.**

LangGraph · interrupts G5 · simulated readback

Typed, current, feasible: three separate claims

Do not collapse the gates

Check	A pass does not prove
Schema ✓	Correct intent interpretation
Binding ✓	Source authenticity
Engineering ✓	Adequate model or field authorization

The solve converged at 112 A; the limit is 100 A. Verdict?

Reject that candidate under the declared current limit.

> **Passing one check does not establish the others.**

Tutorial binding and operator checks

A vote cannot raise a declared limit

A useful failure test

Can a majority vote make this candidate acceptable?

No. Reject it, preserve the incumbent, and record the failed margin.

Two models favor a candidate

The numerical check reports 112 A

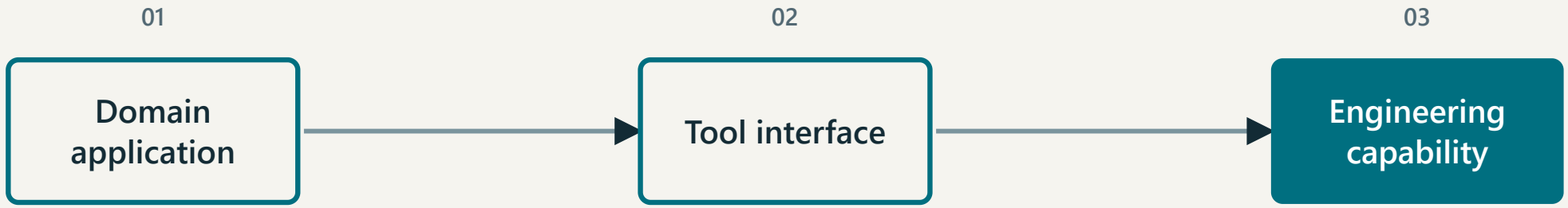
The declared limit remains 100 A

> **Selection chooses a trial; verification determines whether it is admissible.**

Tutorial candidate verification Grid-Agent · sandbox validation

PowerAgent: connect engineering capabilities

Research lens · interoperability



PowerMCP exposes engineering tools

PowerWF organizes reusable workflows

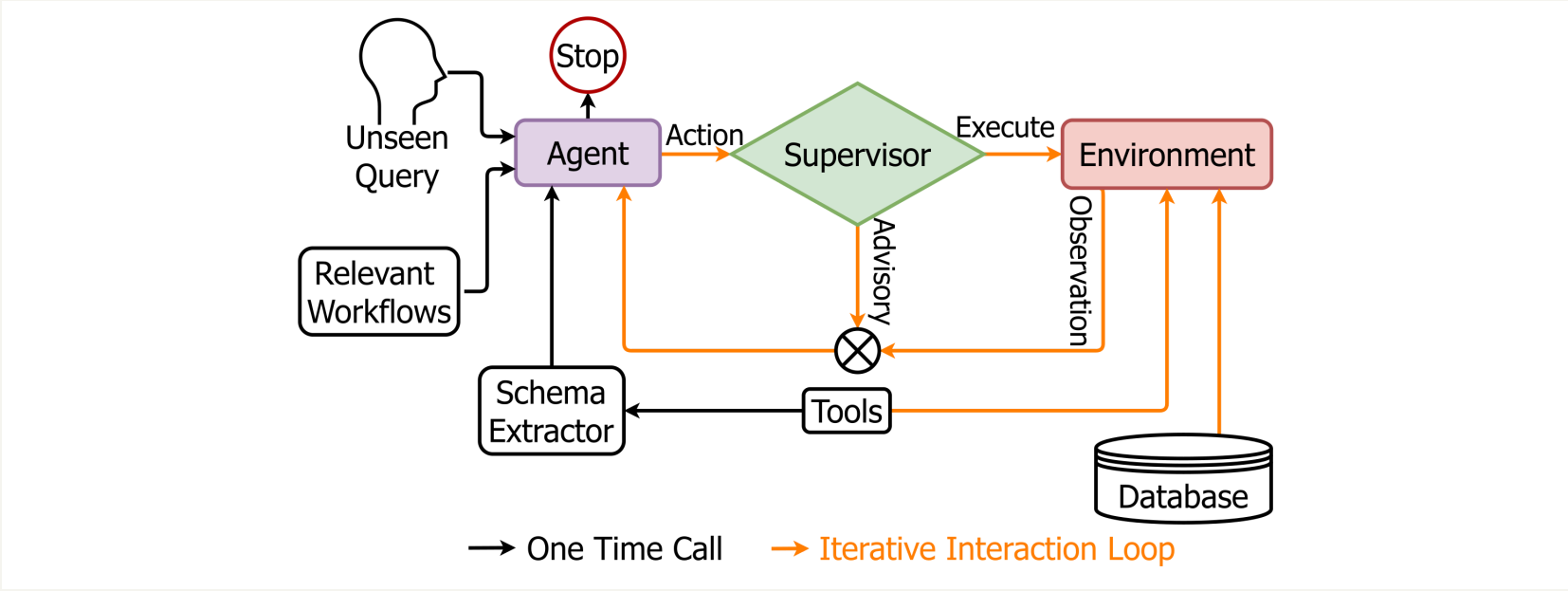
Capabilities vary with the selected application

> **Reusable tool interfaces and workflows reduce integration effort across applications.**

Zhang & Xie · PowerAgent · IEEE PEM 2025 PowerAgent · official ecosystem PowerMCP

PowerDAG: check prerequisite relationships

Research lens · dependency supervision



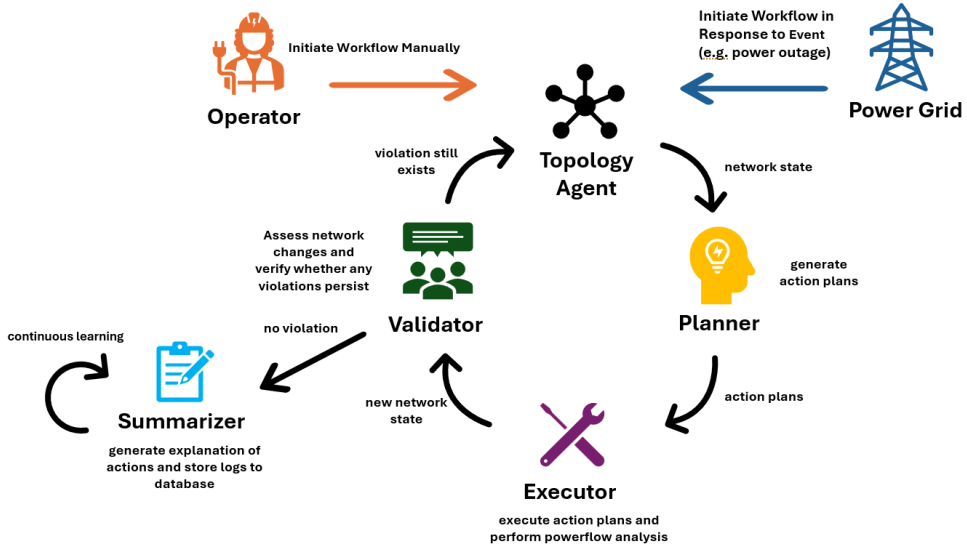
Badmus & Pandey (2026), Fig. 2, CC BY-SA 4.0. Unchanged figure.

- Retrieve relevant workflow exemplars
- Check prerequisite tool writes within covered scope
- One-shot advisory; repeated calls may proceed

A one-shot advisory detects a covered prerequisite violation; repeated calls may proceed.

Grid-Agent: test a change before retaining it

Research lens · sandbox and rollback



Zhang et al. (2025), Grid-Agent v3, Fig. 1, CC BY 4.0. Unchanged.

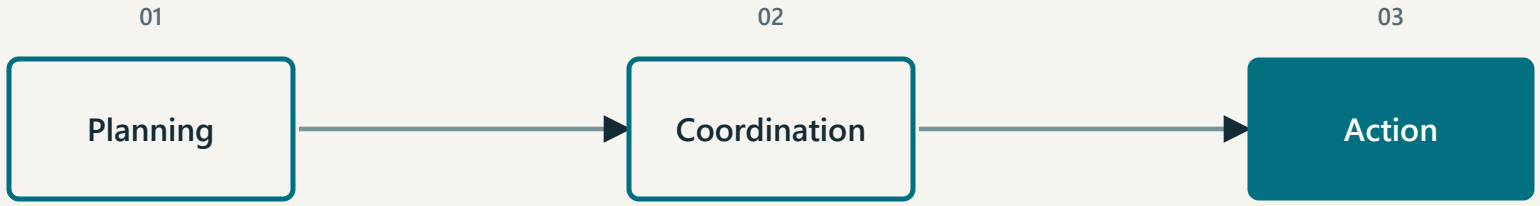
- Topology, planner, executor, validator, summarizer
- Evaluate actions in a simulation sandbox
- Retain or roll back using returned evidence

> **A failed trial should not silently replace the incumbent.**

Zhang et al. · Grid-Agent · v3, 2025 preprint Figure and reuse license

X-GridAgent: separate planning, coordination, and action

Research lens · hierarchy



Planning decomposes the requested analysis

Coordination routes and tracks tasks

Action selects domain tools and reads results

> **A hierarchy helps organize work only when each handoff preserves the required context.**

Wen & Chen · X-GridAgent · 2025 preprint

Read the ticks as scope, not a league table

✓ documented · △ qualified · NR not reported

Work	Tools	Feedback	Checks	Review
PowerAgent ecosystem	✓	✓	△	✓
PowerDAG	✓	✓	△	NR
Grid-Agent v3	✓	✓	✓	△
X-GridAgent	✓	✓	△	△
PowerAgentBench-SS	✓	✓	✓	△
Tutorial	✓	✓	✓	✓

Different tasks, models, budgets, and evaluation boundaries

Benchmark work evaluates agents; it is not a controller

Our benefit must be tested against matched baselines

> **Cross-paper capability marks do not establish a common performance ranking.**

Qualified matrix and source-by-source evidence PowerAgentBench-SS · 2026 preprint

Our first experiment makes every transition visible

Hands-on · Graph_Workflow_Lab.ipynb

$$i = 0.4 + 0.8u, \qquad i \leq L$$

u Dimensionless command i Normalized illustrative current L Declared toy limit

Simulate the command

Verify the limit

Repair by 0.25 or hold

> **Learn real graph execution with deliberately simple arithmetic.**

G0–G5 · executed graph lab

G0: confirm the runtime and the experiment boundary

Run the setup cell

Graph_Workflow_Lab.ipynb · G0 · lines 5–13

```
os.environ["LANGSMITH_TRACING"] = "false"
os.environ["LANGCHAIN_TRACING_V2"] = "false"
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command
from src.graph_workflow_lab import initial_state, recommendation_id, summary
versions = {name: version(name) for name in ("langgraph", "langgraph-checkpoint", "langchain-core")}
print(versions)
print("Rule-driven toy graph. No LLM, network call or electrical-model validation.")
```

1 Imports the real LangGraph runtime

2 Disables external tracing in this lab

3 Prints versions and the toy scope

> **Record the installed versions before interpreting the execution.**

G0 source and captured execution LangGraph · official overview

G1: turn the record into a state schema

Code ↔ concept

Graph_Workflow_Lab.ipynb · G1 · lines 1–10

```python
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

1
command and limit describe the trial

2
attempts and max_repairs bound retries

3
Annotated[list[str], add] preserves events

After simulate and verify append one event each, how many remain?

Two events; the list-addition reducer keeps both.

> **The schema names fields; the trace annotation selects its merge rule.**

G1 · exact notebook code Python · TypedDict

G2: a node returns only its updates

Simulation then verification

Graph_Workflow_Lab.ipynb · G2 · lines 1–7

```python
def simulate(s):
    value = 0.4 + 0.8 * s["command"]
    return {"current": value, "trace": [f"simulate: {value:.3f}"]}

def verify(s):
    ok = s["current"] <= s["limit"] + 1e-12
    return {"passed": ok, "trace": [f"verify: {ok}"]}
```

1. simulate calculates the toy response
2. verify evaluates the declared inequality
3. Both append a visible event

For u = 0.75 and L = 0.80, does verification pass?

i = 1.00, so the first verification fails.

> **The calculation and the acceptance test are inspectable functions.**

G2 · exact simulate and verify functions

G2: the route is four readable lines

The policy is explicit

Graph_Workflow_Lab.ipynb · G2 · lines 19–22

```python
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["max_repairs"] else "hold"
```

1. Read passed first
2. Otherwise compare attempts with max_repairs
3. Return one permitted node label

> **A successful test reports; a failed test repairs only while budget remains.**

G2 · exact conditional router

G3: wire the functions into the diagram

Build and compile

Graph_Workflow_Lab.ipynb · G3 · lines 7–14

```
7     builder.add_edge(START, "simulate")
8     builder.add_edge("simulate", "verify")
9     builder.add_conditional_edges("verify", route,
10        {"report": "report", "repair": "repair", "hold": "hold"})
11    builder.add_edge("repair", "simulate")
12    builder.add_edge("report", END)
13    builder.add_edge("hold", END)
14    return builder.compile(checkpointer=checkpointer)
```

1
START enters simulation

2
Verification selects report, repair, or hold

3
Repair returns to simulation

> **Edges permit execution paths; compilation does not prove the engineering assumptions.**

G3 · exact graph wiring LangGraph · graph construction

G4: predict, run, and trace three outcomes

Hands-on · four minutes

Initial u	Limit L	Budget K	Expected result
0.25	0.80	2	Ready · i = 0.60
0.75	0.80	2	One repair · i = 0.80
0.90	0.50	1	One repair, hold · i = 0.92

Which case reaches the repair node but still must hold?

The third: u becomes 0.65, giving i = 0.92 > 0.50.

> **A legitimate workflow can finish ready or hold.**

G4 · executed three-case experiment

G4: the conventional baseline produces the same result

A controlled orchestration comparison

Graph_Workflow_Lab.ipynb · G4 · lines 15–19

```
15 from src.graph_workflow_lab import fixed_baseline
16 for name, args in cases.items():
17     baseline = fixed_baseline(initial_state(*args), simulate, verify, repair,
   route, ready, hold)
18     assert summary(baseline) == summary(results[name])
19 print("Direct Python baseline matches every command, outcome and trace: 3/3.")
```

1
Same simulation and verification functions

2
Same repair rule and stopping policy

3
3/3 outcomes and event histories match

What did the graph add in this experiment?

An explicit execution structure and reusable pause/state mechanisms; it did not improve the toy command.

> **The graph matches all three commands, outcomes, and traces.**

G4 · direct-Python baseline assertion

G5: stop at the pending review node

Pause and inspect

Graph_Workflow_Lab.ipynb · G5 · lines 11–17

```
11 review_app = build_graph(human_review, InMemorySaver())
12 config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex},
   "recursion_limit": 40}
13 pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
14 snapshot = review_app.get_state(config)
15 print("Pending node:", snapshot.next, "| acknowledged:",
   snapshot.values["acknowledged"])
16 print("Interrupt payload:", pause["__interrupt__"][0].value)
17 assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
```

1. Build with a checkpointer
2. Invoke the graph with a thread identifier
3. Inspect the snapshot and interrupt payload

> **Before resumption, the report node is pending and acknowledgment is false.**

G5 · captured pre-resume snapshot LangGraph · persistence

G5: resume with the matching readback

Complete the human-interaction example

Graph_Workflow_Lab.ipynb · G5 · lines 18–21

```
18 response = {"approve": True, "recommendation_id":
   recommendation_id(snapshot.values)}
19 resumed = review_app.invoke(Command(resume=response), config)
20 print("After simulated readback:", summary(resumed))
21 assert resumed["acknowledged"] and review_app.get_state(config).next ==
   ()
```

1. Use the same execution configuration
2. Approve the same recommendation identifier
3. Inspect TOY_ACKNOWLEDGED and the final trace

What should a declined or wrong-identifier response produce?

TOY_HOLD, with acknowledgment false.

> **The scripted acknowledgment changes status without dispatching equipment.**

G5 · simulated readback and final assertion LangGraph · interrupt/resume

L1: change the request; reject the old evidence

Hands-on · open Classroom_Labs.ipynb

Classroom_Labs.ipynb · L1 · lines 1–5

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

1. Run L0, then L1
2. Predict three binding results
3. Explain what matching hashes do not establish

Comparison	Binding
Same request and command	True
Changed load, old evidence	False
Changed command, old evidence	False

> **Evidence consistency must be checked again after a request or command changes.**

L0–L1 · exact binding experiment Tutorial binding implementation

Model-based and model-free methods enter the same workflow

Methods first; use cases next

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) \qquad\qquad u_t = \pi_\theta(o_t)$$

H, k Planning horizon and predicted step x_k, u_k, ℓ State, control, and declared stage cost o_t, π_θ Observation and learned policy with parameters θ t current decision time

Model-based: optimize predicted behavior

Model-free: learn an action rule from experience

Voltage lab: MPC planning and saved SAC inference

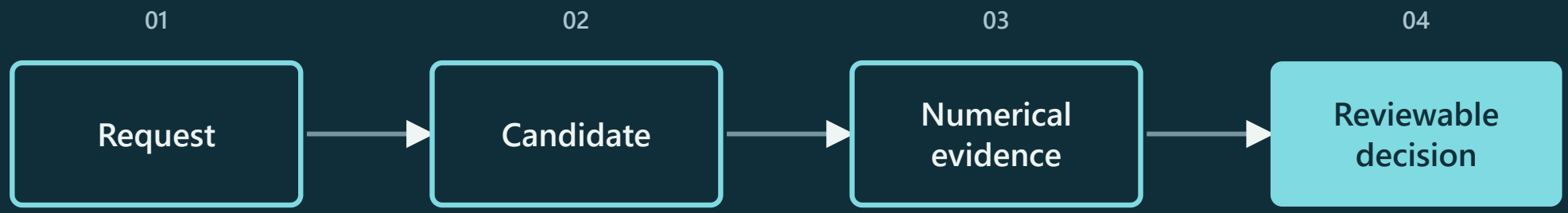
PPO is discussed as an alternative, not trained here

> **MPC and SAC are candidate generators that still face a common verifier.**

Rawlings, Mayne & Diehl · MPC Haarnoja et al. · SAC · ICML 2018 Schulman et al. · PPO · 2017

The graph is ready; now ask a physical question

Next · voltage support



MPC and SAC propose inverter commands

LinDistFlow and OpenDSS expose model-scope differences

Llama proposes bounded follow-up work

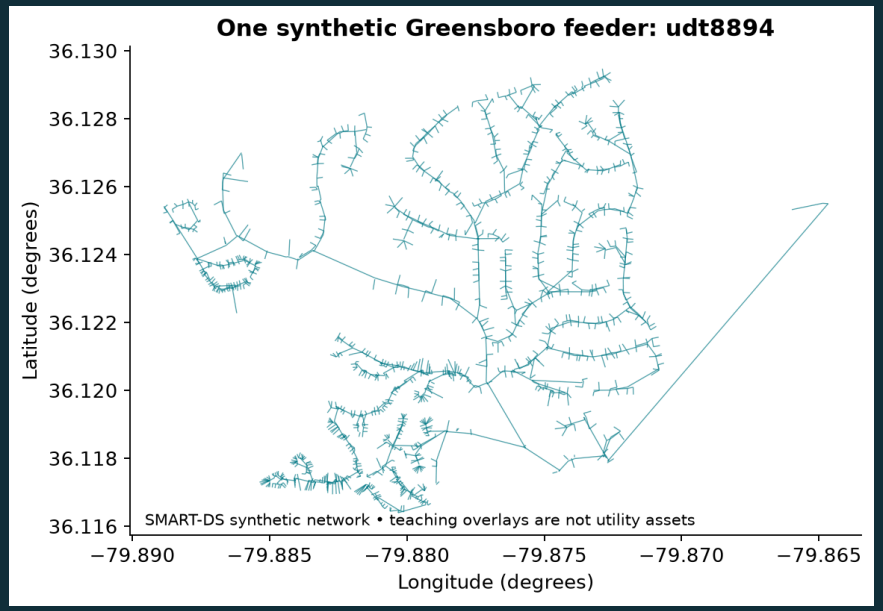
Measure objective, rejected checks, and added time

> **Compare the same scenario, constraints, budget, and evidence boundary.**

L2–L5 · voltage experiments Matched evaluation principle · PowerAgentBench-SS

One feeder. Two controls. One checked recommendation.

40–42 min · Voltage support



SMART-DS synthetic Greensboro feeder uhs20_1247–udt8894. One selected feeder; no live utility deployment.

- Use available PV
- Respect voltage and current limits
- Keep the operator in the decision loop

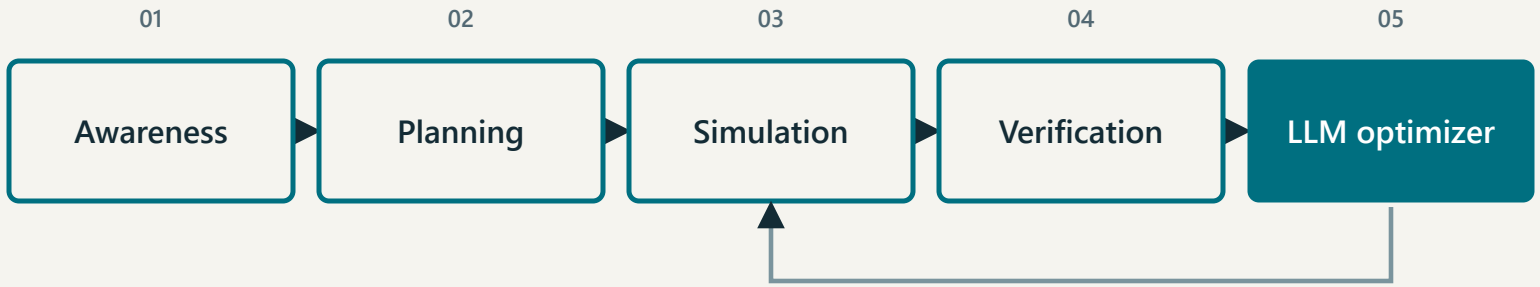
What must the operator know before accepting a new inverter setting?

> **Start with an operating request whose physical scope is explicit.**

SMART-DS · synthetic distribution models Executed tutorial · Classroom Labs

Follow the evidence through all five stages

Workflow map



Awareness → validated request

Planning → MPC and SAC proposals

Simulation and verification → checked evidence

LLM optimizer → bounded next trial

> **Every stage returns an inspectable result, even without an LLM call.**

Rawlings, Mayne & Diehl · MPC Haarnoja et al. · SAC (ICML 2018) Executed tutorial · Classroom Labs

Awareness: make assumptions machine-checkable

Stage 1 · Awareness

Classroom_Labs.ipynb · L1 · lines 1–5

```
request = lab.validate_request(dict(task='voltage', backend='lindistflow3',
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {'control': [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {'example_only': True, 'passed': True})
changed_request = {**request, 'load_scale': 1.10, 'revision': 2}
```

1. Task: voltage support
2. Load multiplier and current derating
3. Model scope and request revision
4. Output: validated contract or rejection

Does validating a request prove that its measurements are true?

No. Schema validity, provenance and physical truth are different properties.

> **A useful request identifies what must be true before planning.**

Executed tutorial · Classroom Labs

Model-based and model-free methods share an action contract

Stage 2 · Planning

Method	Decision-time work	Physical gate
MPC	Model + optimization	Required
SAC	Saved actor inference	Required
PPO	Discussed alternative	Required

MPC: predict with a model, then optimize

SAC: infer with a trained continuous-action policy

PPO: another continuous-action option, discussed only

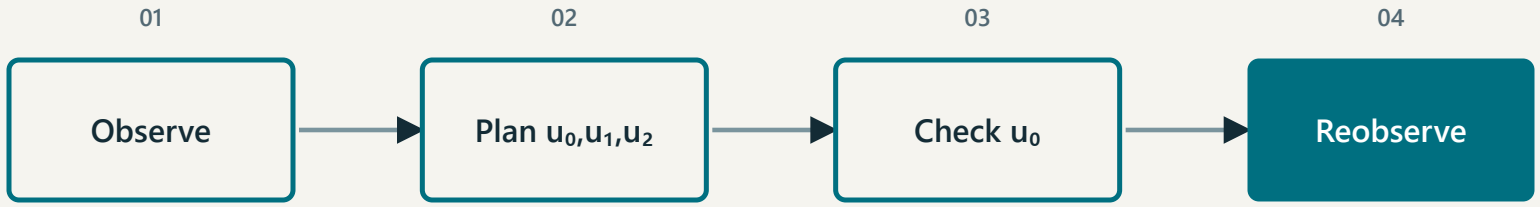
Both: return the proposed command for checking

> **Compare candidate actions under the same scenario, cost and limits.**

Rawlings, Mayne & Diehl · MPC Haarnoja et al. · SAC (ICML 2018) Schulman et al. · PPO (2017)

MPC plans ahead, then uses the first move

Model-based control



Observe the present state

Optimize a sequence over H steps

Test and apply only the first command

Update the forecast and solve again

> **A horizon informs the current action; new observations trigger replanning.**

Rawlings, Mayne & Diehl · MPC

Read the MPC objective from left to right

General formulation → implemented specialization

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k) \quad \text{s.t.} \quad x_{k+1} = f(x_k, u_k, d_k), \quad g(x_k, u_k) \le 0$$

x_k Predicted state at step k

u_k, d_k Control and forecast demand

H, γ Horizon and discount; tutorial H=3, γ=0.98

ℓ, f, g Stage cost, prediction model, constraint function

Minimize discounted stage costs

Enforce the prediction model

Keep every planned step inside constraints

> **Prediction and constraints are part of the optimization problem.**

Rawlings, Mayne & Diehl · MPC

Two numbers specify the inverter fleet command

Control vocabulary

$$u = (a, b), \qquad p_i = aS_i, \qquad q_i = bS_i$$

u Two-number fleet command

a, b Dimensionless active and signed reactive participation

S_i Inverter i apparent-power rating, kVA

p_i, q_i Active kW and reactive kvar commands

a sets active participation

b sets signed reactive participation

Negative b means reactive absorption

Every inverter retains its own rating

> **Define units and signs before interpreting any numerical result.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection

Four cost terms express four preferences

Objective, not a safety certificate

$$\ell = (1 - a)^2 + 0.05b^2 + 10\,\mathrm{mean}[(V - 1.01)^2] + 0.05\|u - u_{\mathrm{prev}}\|_2^2$$

V Modeled voltage magnitudes, p.u. u_{prev} Previous two-number command ℓ Dimensionless weighted teaching cost

Preserve active capability

Limit reactive effort

Keep voltages near 1.01 p.u.

Avoid abrupt changes from the previous command

> **A cheaper reactive command can still worsen the total outcome.**

Recorded experiment · LLM response selection

Feasibility comes before ranking

Stage 4 · Verification preview

$$V_{\min} \le |V_i| \le V_{\max}, \quad |I_e| \le I_e^{\max}, \quad a^2 + b^2 \le 1, \quad 0 \le a \le 1$$

V_i Bus-phase voltage, p.u. $I_e, I_e^{\max}$ Branch current and allowed current, A a, b Normalized active and reactive command

Network: voltage and current bounds

Devices: inverter capability and action bounds

Evidence: finite values and completed solve

Identity: matching request, command and model

> **Neither a low cost nor a majority vote can waive a limit.**

Rawlings, Mayne & Diehl · MPC Recorded experiment · LLM response selection

The two backends see different constraint sets

Stage 3 · Simulation

Model	Covered network	Meaning of PASS
LinDistFlow3	MV + ideal LV aggregation	Reduced contract
OpenDSS	Supplied MV/LV + transformers	Full snapshot contract

LinDistFlow3: MV network, ideal aggregated LV demand

OpenDSS: supplied MV/LV feeder and transformers

Same action can pass one contract and fail another

Changed coverage prevents an accuracy-only comparison

> **Declare model coverage before comparing outcomes.**

Gan & Low · three-phase optimization (2014) EPRI · OpenDSS documentation Executed tutorial · Classroom Labs

LinDistFlow3 retains three-phase coupling

Python model used in the notebook

$$\mathbf{w}_j = \mathbf{w}_i - 2\,\mathrm{Re}\left[\overline{\boldsymbol{\beta}} \odot \left(Z_{ij}\left(\boldsymbol{\beta} \odot \overline{\mathbf{S}}_{ij}\right)\right)\right]$$

$\mathbf{w}_i, \mathbf{w}_j$ Parent and child squared phase voltages, V²

$Z_{ij}, \mathbf{S}_{ij}$ Coupled impedance Ω; downstream phase powers VA

$\boldsymbol{\beta}$ Nominal phase ratios (1, $e^{-j2\pi/3}$, $e^{-j4\pi/3}$)

$\bar{\cdot}, \odot, \mathrm{Re}$ Conjugate, entrywise product, real part

Propagate squared voltages down the radial feeder

Use phase impedance matrices

Assume nominal phase-angle ratios

Report losses diagnostically; omit them from branch balance

> **The linear relation still contains mutual impedance between phases.**

Gan & Low · three-phase optimization (2014) Executed tutorial · Classroom Labs

SAC balances task reward and exploration

Model-free continuous control

$$\max_{\pi} \; \mathbb{E}\left[\sum_{t\geq 0} \gamma^t \{r(s_t, u_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\}\right]$$

s_t, u_t Observation and continuous action

π, r Actor policy and task reward

$\mathcal{H}, \alpha$ Policy entropy and exploration weight

γ Reward discount factor

$\mathbb{E}, t$ Expectation over policy trajectories; discrete training step

Actor maps observations to continuous actions

Reward evaluates the task outcome

Entropy encourages exploration during learning

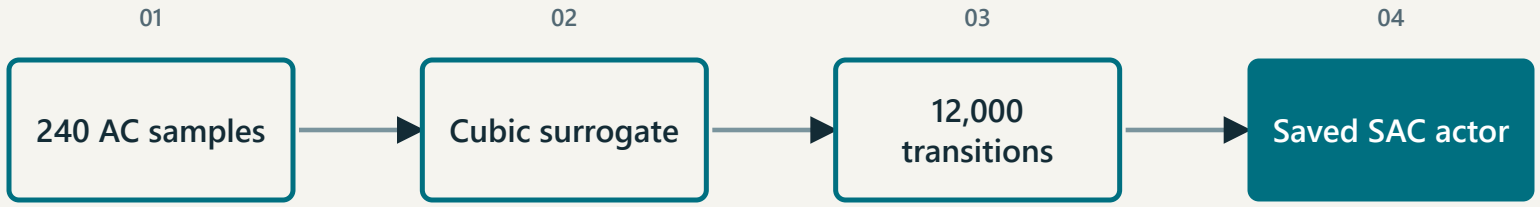
The frozen actor still needs fresh verification

> **A reward penalty is not a proof of constraint satisfaction.**

Haarnoja et al. · SAC (ICML 2018) Schulman et al. · PPO (2017)

Read the training provenance, not only the actor output

SAC artifact used by L2



240 AC solves fit a cubic surrogate

48 separate samples evaluate the surrogate fit

12,000 policy-training transitions use that surrogate

L2 performs new inference with saved weights

> **Training cost and inference cost belong to different measurements.**

Haarnoja et al. · SAC (ICML 2018) Executed tutorial · Classroom Labs

L2: the code changes a scenario, then recomputes

Notebook checkpoint · L2

Classroom_Labs.ipynb · L2 · lines 1–7

```
1 LOAD_SCALE = 1.0
2 RATING_FRACTION = 0.80
3 started = time.perf_counter()
4 voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
5 timings['L2 fresh MPC + SAC + checks'] = time.perf_counter() - started
6 print(voltage['mode'])
7 display(voltage['table'])
```

1. Change one scenario parameter
2. Call the shared experiment helper
3. Time the whole numerical operation
4. Inspect the returned comparison table

Which output would show that a faster proposal was unusable?

> **One cell solves MPC, infers SAC and checks both candidates.**

Executed tutorial · Classroom Labs

Lab: predict, change demand, explain the outcome

53–60 min · 7-minute paired exercise

Classroom_Labs.ipynb · L2 · lines 1–4

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
```

1. Predict before running
2. Run L2 at the default load
3. Change LOAD_SCALE once and rerun L2
4. Record feasibility, objective and the limiting margin

At higher demand, will PV curtailment necessarily increase?

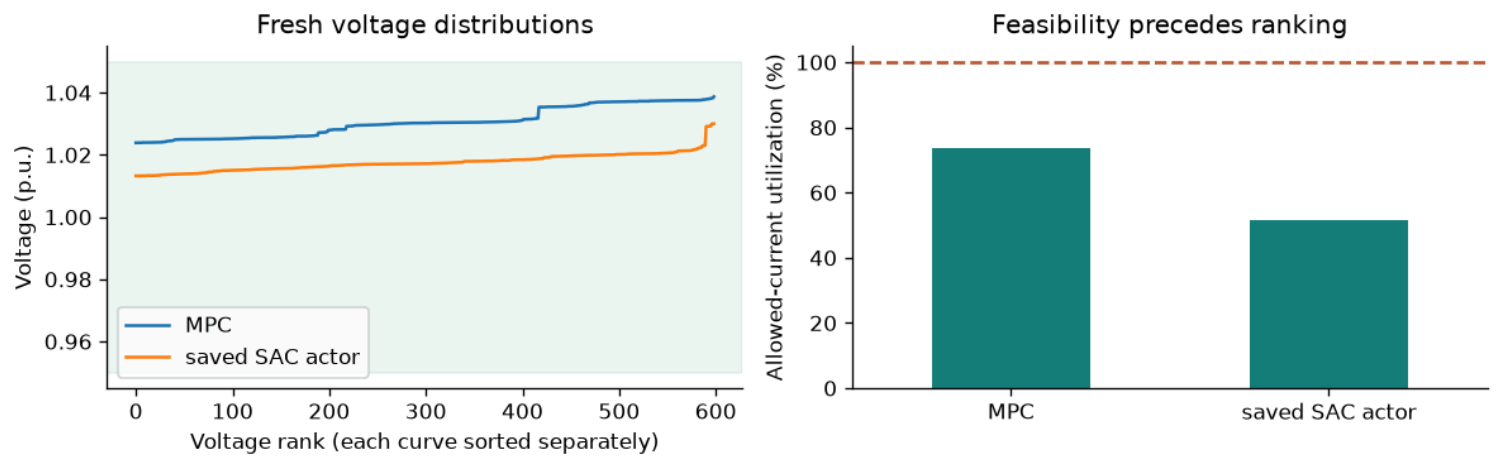
Not necessarily. Local voltage and current constraints can move in different directions.

> **Change one assumption and support your conclusion with a metric.**

Executed tutorial · Classroom Labs

Inspect the margins before the winner label

Fresh L2 default output



Executed L2 default: fresh MPC solve, saved SAC actor inference and reduced-model checks. Sorted curves are voltage ranks.

- MPC objective: 0.031104
- SAC objective: 0.436428
- Both pass the reduced-model gate in this default case
- Voltage curves are separately sorted distributions

Does the lower objective identify which stage caused the improvement?

> **A feasible action can still be much worse on the declared objective.**

Executed tutorial · Classroom Labs

L3: inspect actual archived model responses

Notebook checkpoint · L3

Classroom_Labs.ipynb · L3 · lines 1–8

```
pool = lab.saved_pool('test_01')
display(pd.DataFrame(pool['responses'])[['model_id', 'operation_id', 'reason',
'seconds', 'source']])
display(pd.DataFrame([{'operation': ', '.join(c['operation_ids']),
    'votes': c['vote_count'], 'share': c['vote_share'], 'parameters':
c['operation']['parameters']}
    for c in pool['pool']['candidates']]))
print('ARCHIVED ACTUAL LLM; original context:', pool['context']['id'])
print('Distinct physical candidates:', len(pool['pool']['candidates']))
print('Malformed/duplicate votes rejected:', len(pool['pool']['rejected']))
```

1. Load original test_01
2. Display model, operation, reason and recorded seconds
3. Count distinct physical candidates
4. Keep this context separate from modified L2

> **Recorded model output is evidence to inspect, not a fresh measurement.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection

Three replies produce two different physical trials

Archived actual responses · original test_01

Recorded model	Selected operation	Call time
Llama 3.1 8B	Increase a by 0.0025	1.867 s
Llama 3.2 3B	Reduce |b| by 10%	0.905 s
Qwen 2.5 7B	Reduce |b| by 10%	1.844 s

Qwen wrote “may help improve voltage stability.” What evidence supports that?

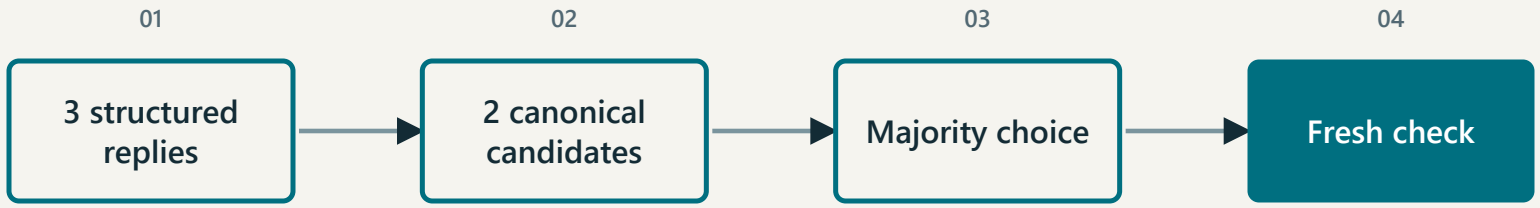
These steady-state tests do not establish voltage stability.

> **A persuasive explanation can contain an unsupported engineering claim.**

Recorded experiment · LLM response selection

Vote on a canonical operation

Stage 5 · LLM optimizer



Parse structured tool and parameter fields

Deduplicate identical physical commands

Count votes with a fixed tie rule

Simulate and verify the selected trial

> **Voting selects the next trial; verification controls whether it is retained.**

Recorded experiment · LLM response selection

LinUCB learns which operation is worth testing

Learned response selection

$$q(a \mid s) = \theta^\top \phi(s,a) + \beta\sqrt{\phi(s,a)^\top A^{-1}\phi(s,a)}, \qquad a^* = \arg\max_a q(a \mid s)$$

s, a Evidence context and canonical operation; a is not inverter participation here

ϕ, θ Context-operation features and learned coefficients

A, β Feature-information matrix and exploration weight

q, a^* Training selection score and highest-scoring permitted operation

Features combine context and candidate operation

Training score includes an exploration bonus

Feedback updates the linear reward model

Frozen evaluation uses predicted reward only

> **The learner chooses among generated operations, not which LLM to call.**

Li et al. · LinUCB (WWW 2010) Recorded experiment · LLM response selection

Learning feedback must follow the physical gate

Reward and evaluation protocol

$$r = \begin{cases} \min\left(1, \dfrac{\max(0, J_{\text{inc}} - J_{\text{trial}})}{0.01}\right) - 0.01, & \text{PASS} \\ -1.01, & \text{rejected} \end{cases}$$

r Post-trial learning feedback $J_{\text{inc}}, J_{\text{trial}}$ Incumbent and trial current-step costs

24 training, 8 validation, 12 test contexts

Five training orders; selected feedback only

Frozen validation and test: zero updates

The 0.01 trial charge is not seconds

> **A predicted reward cannot override a failed physical check.**

Li et al. · LinUCB (WWW 2010) Recorded experiment · LLM response selection

L4: test the frozen choice, not the predicted reward

73–76 min · 3-minute experiment

Classroom_Labs.ipynb · L4 · lines 1–8

```
decision = pool['selection']
print('Frozen:', pool['frozen'], '| selected-feedback training updates:', pool['training_updates'])
print('Bandit operation:', decision.operation_id, '| predicted reward:', round(decision.predicted_reward, 4))
print('Same canonical choice as majority:', decision.candidate_id == pool['vote_id'])
started = time.perf_counter()
selected_trials = lab.trial_saved_selection(pool)
timings['L4 new verification of archived choices'] = time.perf_counter() - started
display(selected_trials['table'])
```

1. Run L4
2. Compare the learner choice with the majority vote
3. Inspect fresh feasibility and cost
4. Explain what happens after a failed trial

Predicted reward is positive, but the current limit fails. Accept?

Reject. The gate takes precedence over reward prediction.

> **The selected action must earn a new numerical result.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection

A useful experiment can find no learned advantage

Held-out MV pilot · 12 scenarios

MV held-out method	Mean J	Full-feeder AC PASS
Fixed refinement	0.03400656	0 / 12
Majority vote	0.03398479	0 / 12
Frozen learner	0.03398479	0 / 12

Which ablation would test whether language-model calls are necessary?

Fixed refinement mean J: 0.03400656

Vote and learned selector mean J: 0.03398479

Vote and learner tie on every held-out case

A posthoc fixed Q-reduction rule reproduces their choices

> **Compare extra agents with a strong, inexpensive deterministic supervisor.**

Recorded experiment · LLM response selection

Account for the full decision path

Latency is part of the result

MV held-out method	Accounted serial time
Fixed refinement	3.327 s
Frozen learner with response pool	8.119 s

What would change if the local model had to load from disk?

Fixed-refinement accounted serial time: 3.327 s

Learned-selection accounted serial time: 8.119 s

All pool selectors pay for all three model calls

Separate setup, inference, simulation and verification

> **Fast actor inference does not imply a fast end-to-end recommendation.**

Recorded experiment · LLM response selection Ollama · chat API and timing fields

L5: audit the same action, then replan separately

Notebook checkpoint · L5

Classroom_Labs.ipynb · L5 · lines 1–6

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings['L5 selected model scope checks'] = time.perf_counter() - started
display(scope_audit['table'])
print(scope_audit['optional_mode'])
print('An MV-only PASS cannot acknowledge a request requiring full MV/LV coverage.')
```

1. Default: fresh MV check
2. Optional: same-action full-feeder OpenDSS audit
3. Optional: separate MPC solve using the AC backend
4. Keep the frozen selector unchanged

> **Cross-model auditing and replanning answer different questions.**

Executed tutorial · Classroom Labs EPRI · OpenDSS documentation

Broader coverage reveals a missing constraint boundary

Recorded paired pilot · 12 cases

Final learned-workflow actions	Selected-model gate	Full-feeder AC audit
MV planning	12 / 12	0 / 12
OpenDSS planning	12 / 12	12 / 12

Which component supplied the extra physical information?

The broader numerical model and its acceptance contract.

> **The reduced-model PASS does not satisfy a full-feeder request.**

Recorded experiment · LLM response selection EPRI · OpenDSS documentation

Verification returns reasons, not just a green badge

Stage 4 · Verification

01 Recompute the exact selected action

02 Check voltage, current and capability limits

03 Bind results to the request and candidate

04 Return PASS, rejection reasons and model scope

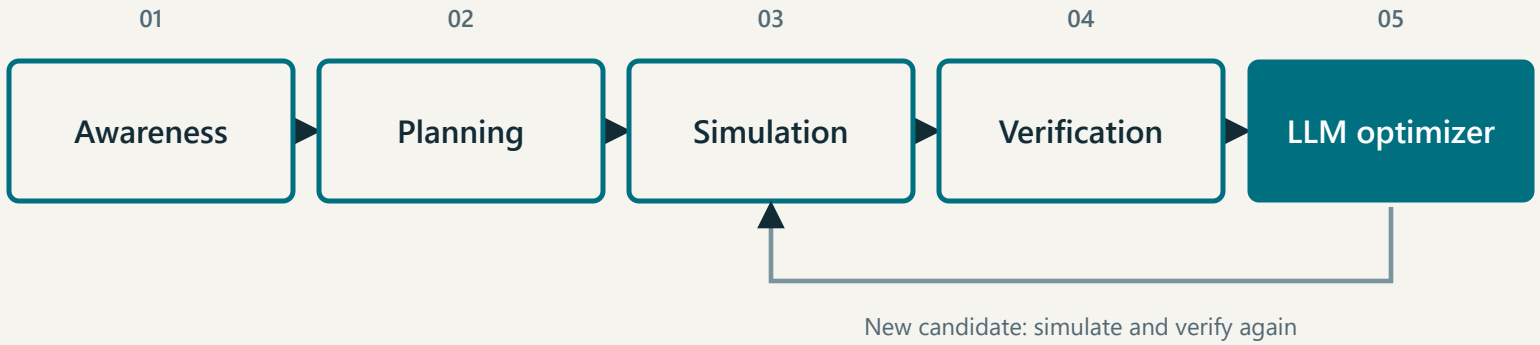
Which fields would you want beside a rejected recommendation?

> **A rejected recommendation should explain the failing contract.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection

Voltage support: keep the benefit claim precise

80–82 min · Handoff before the break



Awareness: validated scenario

Planning: MPC and SAC candidate controls

Simulation and verification: scope-specific physical evidence

Optimizer and operator: bounded refinement, review and readback

> **The workflow exposes assumptions and failures; better control must be measured.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection

PAUSE

Ten-minute break

82–92 minutes

01 Save the voltage experiment.

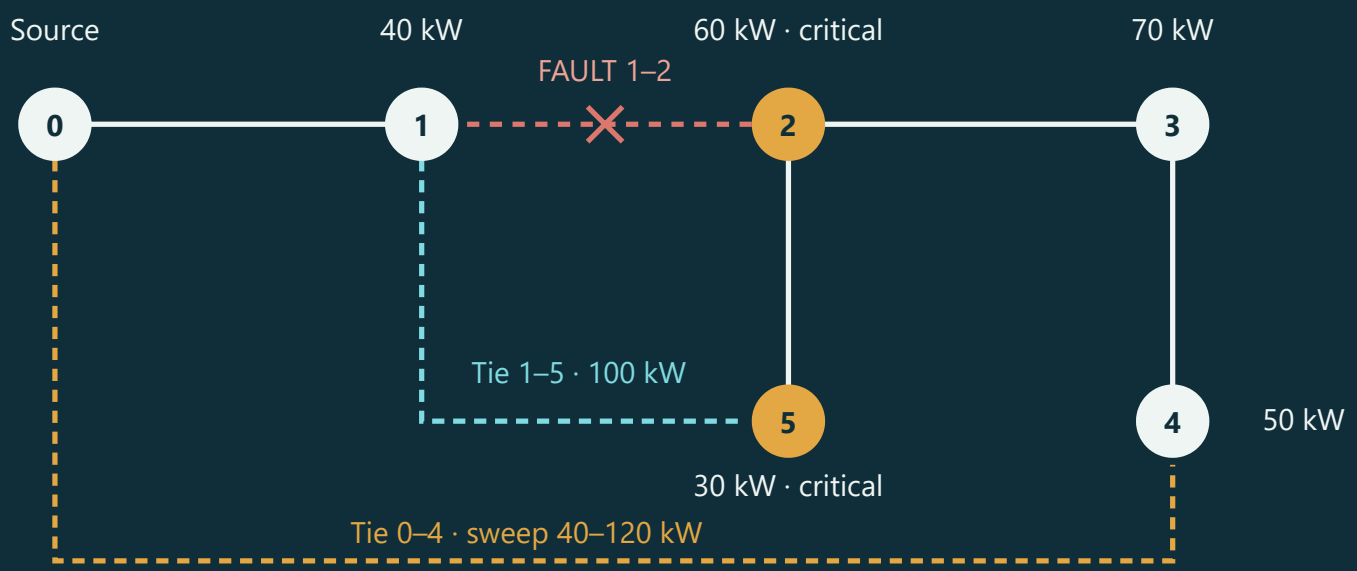
02 Return with one model-mismatch question.

> **Next we change the decision from continuous inverter control to discrete restoration.**

Classroom_Labs L2–L5

After the fault, decide what can be reconnected

92 min · Restoration



Available network options, not a solved switching plan · kW transport model

One faulted edge is unavailable

Two possible ties reconnect downstream buses

Critical and ordinary loads compete for capacity

Restoration chooses topology and load pickup under operating constraints.

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

Three binary decisions describe one restoration plan

Stage 1 · Awareness → decision vocabulary

s_i

Binary load pickup

s_i: pick up load i

y_i

Binary energized bus

y_i: energize bus i

z_e

Binary closed edge

z_e: close edge e

A fault exclusion is a hard constraint

> **Separate pickup, energization and switch closure.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

Choose a service objective that matches the instruction

Stage 2 · Model-based planning

$$\max \sum_i w_i d_i s_i, \qquad s_i \le y_i, \qquad z_{ij} \le y_i,\ z_{ij} \le y_j, \qquad z_{\text{fault}} = 0$$

d_i Load demand, kW w_i Priority weight; critical 10, otherwise 1 s_i, y_i, z_{ij} Binary pickup, energized bus and closed edge

Maximize weighted served demand

Critical weights are 10; other weights are 1

Use unit weights for the total-service baseline

Report both critical and total served kW

> **Priority weights express policy; they are not an LLM discovery.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

Power must reach every served load

Physical transport constraints

$$\sum_{e} A_{ie} P_e = d_i s_i \quad (i \neq 0), \qquad -\overline{P}_e z_e \leq P_e \leq \overline{P}_e z_e$$

A_{ie} Signed node-edge incidence entry P_e Active branch flow, kW $\overline{P}_e$ Branch capacity, kW $i = 0$ Source bus

Balance active power at each non-source bus

Limit flow by the closed edge capacity

An open edge carries no power

The source supplies the served demand

> **An attractive pickup list still needs a feasible source path.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

Radiality needs connectivity as well as an edge count

Graph certificate

$$\sum_e A_{ie} f_e = y_i \ (i \neq 0), \qquad |f_e| \leq 5 z_e, \qquad \sum_e z_e = \sum_i y_i - 1$$

f_e Fictitious connectivity flow, not electrical power

5 Maximum number of energized non-source buses

$\sum z_e = \sum y_i - 1$ Tree edge count, used together with connectivity

Send one fictitious unit to each energized non-source bus

Allow that commodity only on closed edges

Combine connectivity with the tree edge count

The commodity is a proof device, not electricity

> **A tree equation alone does not exclude a disconnected island.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

Compare objectives on the same feasible network

Baseline design

Property	Total-service baseline	Priority policy
Same fault and capacities	✓	✓
Same feasible network	✓	✓
Objective	Total kW	Weighted kW
Exact method	Enumeration	MILP + enumeration

If critical service improves after changing weights, what caused that change?

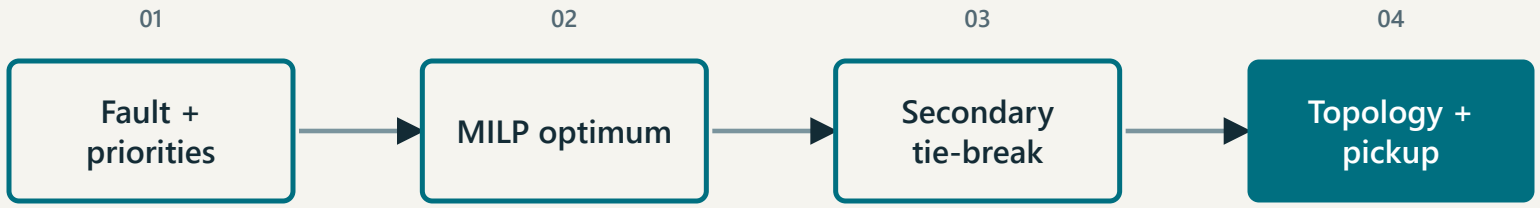
The documented objective policy, unless another controlled experiment shows otherwise.

> **Hold the fault, capacities and constraints fixed.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020) Executed tutorial · Classroom Labs

The planner returns a topology plus a pickup decision

Stage 2 · Standard optimization method



Solve the weighted binary restoration problem

Preserve the best weighted service in a second solve

Minimize energized edges, then apply a small tie-break

Return decisions and solver evidence

> **Use the method that fits the decision type.**

Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020) Executed tutorial · Classroom Labs

L6: a capacity sweep is one reproducible experiment

Notebook checkpoint · L6

Classroom_Labs.ipynb · L6 · lines 1–5

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

1. Declare the capacity levels
2. Solve weighted and unweighted service problems
3. Check the MILP using independent enumeration
4. Display total and critical service

> **Recompute the plan at every tested rating.**

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018)

Lab: tighten the tie and follow the source path

102–110 min · 8-minute paired exercise

Classroom_Labs.ipynb · L6 · lines 1–3

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
```

1. Predict service at 40, 60, 80 and 120 kW
2. Run L6 and add one tighter capacity
3. Compare critical and total service
4. Explain one plateau using the selected loads

Will total service vary smoothly as tie capacity changes?

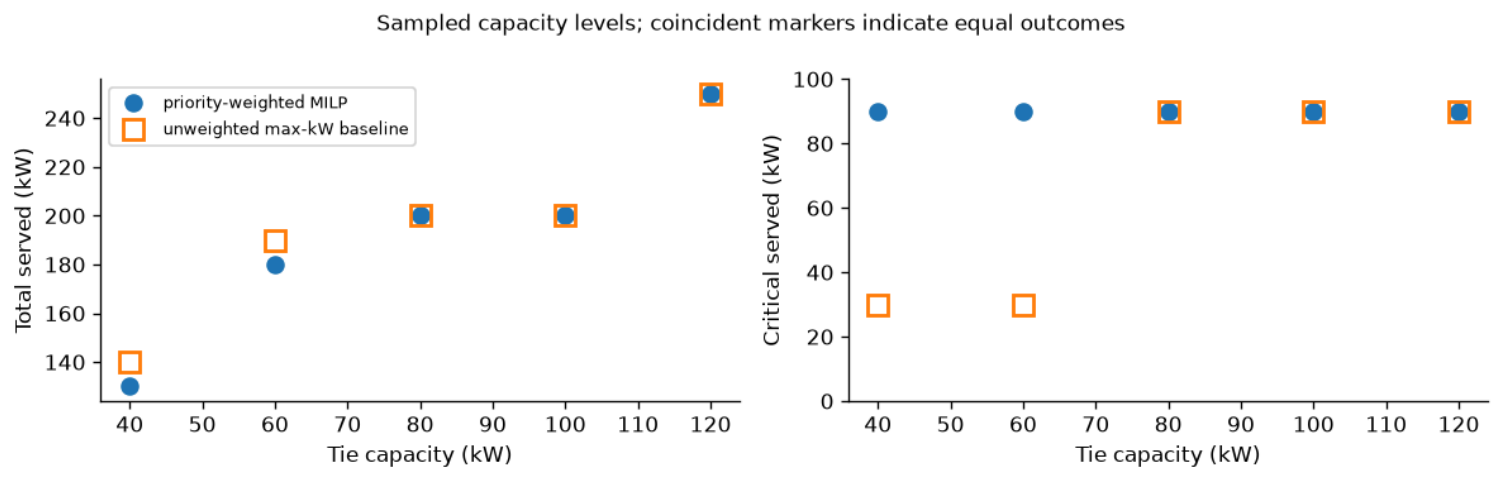
No. Binary load pickup creates thresholds and plateaus.

> **Explain discrete service changes using topology and capacity.**

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018)

The service curve reveals the policy tradeoff

Fresh L6 default sweep



Executed L6 sweep. Markers denote sampled capacities; coincident outcomes overlap. Critical priorities are teaching labels.

- Each marker is a solved capacity level
- Coincident markers mean equal outcomes
- Priority can reduce total kW while preserving critical kW
- Flat regions reflect discrete choices

Read critical service and total service together.

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018)

Case A: 40 kW capacity makes priorities visible

A constrained restoration case

40 kW tie case	Total served	Critical served
Unweighted baseline	140 kW	30 kW
Priority-weighted policy	130 kW	90 kW

Which plan is better if the operator's instruction changes?

Weighted policy: 130 kW total

Weighted policy: 90 kW critical

Unweighted baseline: 140 kW total

Unweighted baseline: 30 kW critical

> **The best total-service plan need not restore the most critical demand.**

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018)

Case B: more capacity can leave service unchanged

A second restoration condition

Why can a 20 kW rating increase restore no additional load?

The next discrete pickup may remain infeasible, or another constraint still binds.

At 80 kW: weighted service is 200 kW

At 100 kW: weighted service remains 200 kW

Both serve 90 kW critical demand

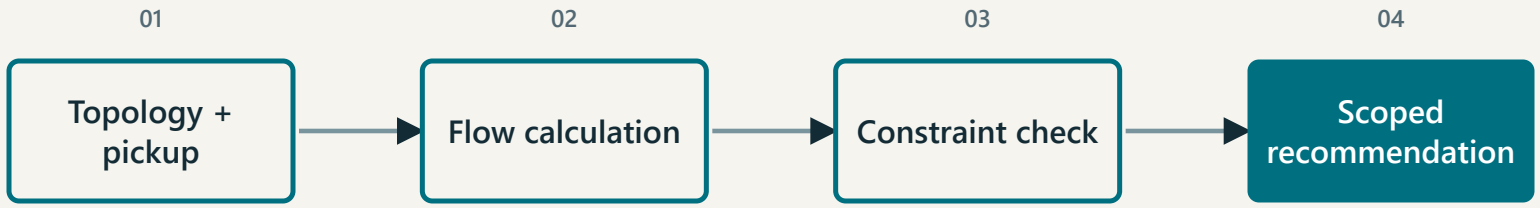
Inspect discrete pickup and the binding path

> **The next available load may need more capacity than the increment supplies.**

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018)

Simulation and verification check different properties

Stages 3 and 4 · Restoration



Simulation: compute flows for the chosen topology

Verification: capacity, fault exclusion and source connectivity

Independent check: enumeration agrees with the MILP

Scope: no AC voltages or switching chronology

> **A candidate topology needs recomputation and an independent contract check.**

Executed tutorial · Classroom Labs Chen et al. · restoration optimization (2018) Wang et al. · radiality constraints (2020)

L7: changing the rating invalidates old evidence

Notebook checkpoint · L7

Classroom_Labs.ipynb · L7 · lines 2–9

```
rest_request = {'task': 'restoration', 'capacity_kw': CAPACITIES_KW[0],
                'critical_nodes': [2, 5], 'model': 'six_node_active_transport',
'revision': 1}
rest_candidate = {'pickup': rest_record['milp']['pickup'], 'closed_edges':
rest_record['milp']['closed_edges']}
rest_evidence = lab.bind_evidence(rest_request, rest_candidate,
    {'objective': rest_record['milp']['objective'], 'enumeration_agrees':
rest_record['agreement']})
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate,
rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request,
rest_candidate, rest_evidence))
```

1. Bind the original plan to its request
2. Change capacity and revision
3. Test the old evidence against the new request
4. HOLD and request a fresh constrained solve

Can we keep the old green status after lowering the rating?

No. The changed contract requires new evidence.

> **A correct old solution is not a checked solution for a new request.**

Executed tutorial · Classroom Labs

Inspect all five stages in a recorded feeder run

Archived actual workflow · inspected by L10

Classroom_Labs.ipynb · L10 · lines 16–23

```
16 from IPython.display import HTML
17 for case, backend in [('voltage', 'lindistflow3'), ('restoration', 'lindistflow3'),
   ('cyber_stealth', 'dc')]:
18     full_trace = lab.archived_stage_table(lab.archived_stage_replies(case,
   backend))
19     assert full_trace.stage.nunique() == 5
20     display(HTML('<details open><summary>ARCHIVED ACTUAL: ' + case +
21         ' — all five stages, every recorded event</summary>' +
22         full_trace.drop(columns='role_group').to_html(index=False, escape=True) +
   '</details>'))
23 print('Selected returned fields are exact archive values; complete records:
   results/proposal_complete_workflow.json')
```

1
Awareness: fault isolation and equipment permissions

2
Planning: bounded catalog candidates

3
Simulation and verification: power flow and thermal rejection

4
LLM optimizer: permitted shedding refinement

> **The live MILP lab and the recorded feeder catalog are separate experiments.**

Recorded experiment · five-stage workflow Executed tutorial · Classroom Labs

RESTORATION

A recorded Llama ranking needs numerical scrutiny

Actual public responses · Llama 3.1 8B

01 Planning: GENERATE_CANDIDATES · 0.874 s

02 Refinement: SHED_075 · 1.259 s

03 Later ranking lists E02 before E03

04 Recorded costs: −5509.13524 versus −6109.97130

Which is lower: −5509 or −6110?

−6110. The verified numeric objective should control the ranking.

> **Explanations are reviewable claims, not substitutes for sorting verified scores.**

Recorded experiment · five-stage workflow

RESTORATION

The feeder comparison ties on restored service

Archived matched workflow comparison

Archived MV feeder route	Total service	Critical service	Recorded time
Fixed supervisor	1843.614 kW	474.040 kW	0.306 s
Llama supervisor	1843.614 kW	474.040 kW	5.232 s

Which benefit is supported here: more service or a reviewable workflow?

Fixed and Llama routes: 1843.614 kW total served

Both: 474.040 kW critical served

Fixed route: 0.306 s; Llama route: 5.232 s

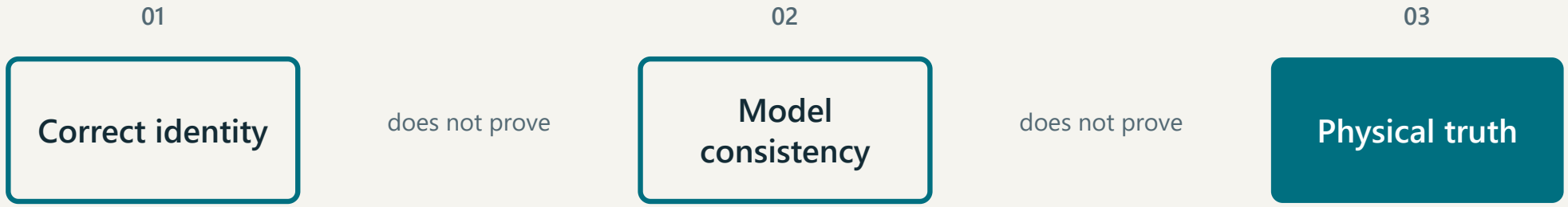
Same selected shedding operation and final model checks

> **Traceability is demonstrated; extra restored load from Llama is not.**

Recorded experiment · five-stage workflow

A matching record can still contain wrong measurements

118–119 min · Transition to cyber response



Correct identifiers bind a record to a request

Consistency checks test agreement with assumptions

Neither property guarantees truthful sensor values

Next: a corrupted estimate that looks consistent

> **Identity, model consistency and physical truth are different properties.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

A controller acts on an estimate, not on hidden truth

119–121 min · Cyber response

01 Four-bus DC teaching system

02 Three unknown non-reference voltage angles

03 Eight flow, injection and angle measurements

04 Two corruption patterns; one controlled experiment

Does a precise-looking state estimate guarantee an accurate state?

> **Start with the information available to the operator.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

WLS combines measurements using their uncertainty

State-estimation vocabulary

$$z = Hx + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R), \qquad \hat{x} = \arg\min_x (z - Hx)^\top R^{-1} (z - Hx)$$

$x, \hat{x}$ True and estimated non-reference angles, rad; dimension 3

z, H Eight measurements and the linear measurement matrix

ϵ, R Measurement noise and its covariance

H maps angles to expected measurements

R weights sensors in their own units

The estimate minimizes weighted mismatch

Observability requires sufficient independent information

> **A model and covariance turn measurements into an estimate.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

A residual tests consistency with assumptions

Stage 1 · Awareness through WLS

$$r = z - H\hat{x}, \qquad J = r^\top R^{-1} r, \qquad J > \chi^2_{5,0.99} = 15.0863 \Rightarrow \text{alarm}$$

r Measurement residual

J Dimensionless residual energy

$\chi^2_{5,0.99}$ 99th percentile with 5 residual degrees of freedom

Residual: unexplained measurement mismatch

Weighted energy J is dimensionless

Eight measurements minus three states gives five degrees

99% threshold: 15.0863

> **No alarm means no detected inconsistency under the stated model.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Case A: bias one measurement

Isolated corruption

01 Change one sensor channel

02 Leave the other channels unchanged

03 Re-estimate the state

04 Inspect the weighted mismatch

Which other measurements could contradict the biased sensor?

> **An isolated error often leaves a residual signature.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Case B: shift measurements along a modeled direction

Coordinated model-consistent corruption

$$a = Hc, \qquad z' = z + a = z + Hc$$

c Chosen state displacement, rad a Coordinated measurement perturbation z' Perturbed measurement vector

Choose a state displacement c

Map it into measurement space using H

Add that pattern to the measurements

Use the unchanged estimator model

> **A structured perturbation can look like a different physical state.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Cancel Hc and see the detector limitation

Same-slide algebra reveal

$$r' = z' - H\hat{x}' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r$$

$\hat{x}' = \hat{x} + c$ Shifted estimate under the same full-rank linear WLS model

$r' = r$ Exactly unchanged residual for this construction

With the same full-rank WLS model: x̂′ = x̂ + c

Substitute into the residual

The added Hc terms cancel

No residual alarm can coexist with a wrong estimate

> **The estimate can move while the residual stays unchanged.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Awareness should preserve provenance and uncertainty

Stage 1 · Cyber workflow

01 Record sensor provenance and trust assumptions

02 Check estimator rank and data validity

03 Report residual alarm status accurately

04 Keep unknown authenticity explicit

Would another LLM reading the same residual add independent sensor evidence?

No. Independent information requires a justified additional source.

> **Do not turn a weak observation into a strong assurance.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

L8: run two corruption families with one shared estimator

Notebook checkpoint · L8

Classroom_Labs.ipynb · L8 · lines 1–5

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

1. Use the same scale grid
2. Compute isolated and coordinated cases
3. Record residual and offline state error
4. Time the whole WLS sweep

> **Sweep the perturbation size and retain comparable conditions.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Lab: separate detector evidence from evaluation truth

128–135 min · 7-minute exercise

Classroom_Labs.ipynb · L8 · lines 1–3

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
```

1. Predict both residual curves
2. Run L8
3. Mark the 15.0863 threshold
4. Explain which plot an operator can actually use

Which curve can be computed without knowing the true state?

The WLS residual curve; state error requires the synthetic truth.

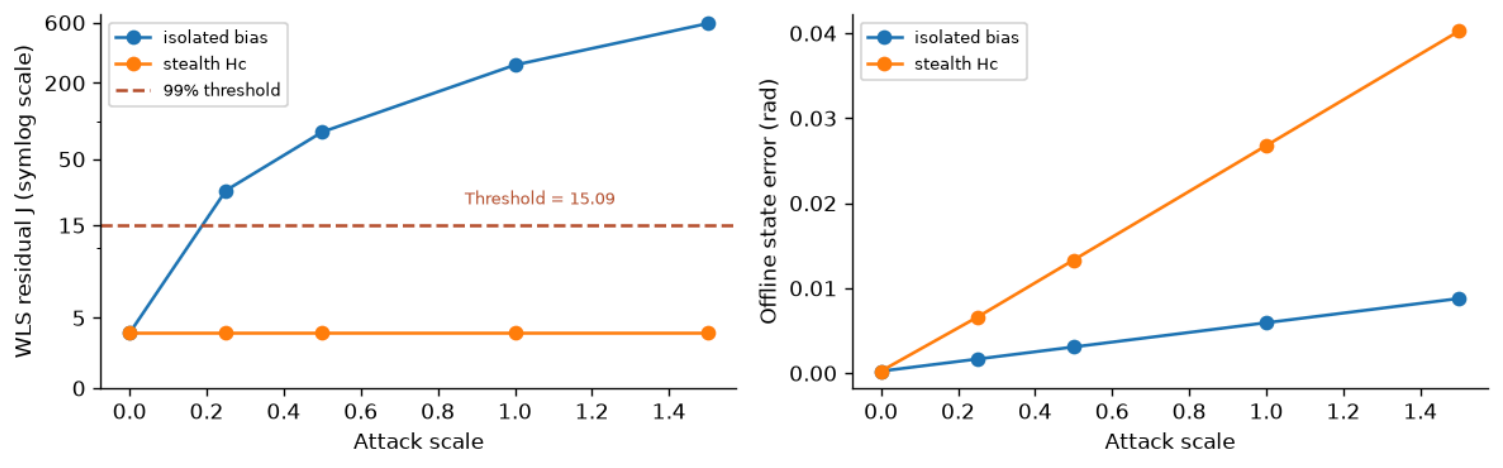
> **Find a case with growing error and no residual alarm.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

CYBER RESPONSE

One curve is flat while the state error grows

Fresh L8 default sweep



Executed L8 sweep. Left: detector-visible residual. Right: offline state error using synthetic truth.

- Coordinated residual J stays near 3.92022
- Isolated scale-1 residual J is about 277.497
- Threshold is 15.0863
- The right-hand error curve uses offline truth

Model consistency and state accuracy answer different questions.

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Plan recovery from a justified information source

Stage 2 · Model-based constrained control

$$\min_{0 \le u \le 0.5} \frac{1}{2} u^2 \quad \text{s.t.} \quad \left| F \hat{x}_{\mathcal{T}} + b_u u \right| \le \overline{f} - 0.01$$

u Added node-2 injection, balanced at source, p.u.

$\hat{x}_{\mathcal{T}}$ State estimated from assumed protected readings

F, b_u Angle-to-flow map and control sensitivity

$\overline{f}$ Branch limits (0.35, 0.18, 0.35, 0.35) p.u.

Estimate from three assumed protected angle readings

Choose the smallest corrective injection

Keep predicted branch flows inside tightened limits

HOLD if protected information is unavailable

Protected measurements change the state information used by the planner.

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Three independent readings remove redundancy

Observability is not authentication

Does full rank imply enough redundancy to test consistency?

$$\operatorname{rank}(H_{\mathcal{T}}) = 3, \qquad m_{\mathcal{T}} - n = 3 - 3 = 0$$

$H_{\mathcal{T}}$ Measurement matrix for the protected subset

$m_{\mathcal{T}}, n$ Protected measurement count and state dimension

No. This exactly determined protected subset has zero residual degrees of freedom.

Three protected angle readings

Three unknown state variables

Rank 3 permits estimation

Residual degrees of freedom: zero

> **A full-rank estimate can still lack a residual consistency test.**

Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

L9: compare trusted evidence with its absence

Notebook checkpoint · L9

Classroom_Labs.ipynb · L9 · lines 1–9

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
display(pd.DataFrame([
    {'condition': 'No trusted replacement channels', 'status': without_trust['status'],
     'u_pu': None, 'offline_actual_violations': None},
    {'condition': 'Three trusted channels assumed', 'status': with_trust['status'],
     'u_pu': with_trust.get('flexible_injection_pu'),
     'offline_actual_violations': with_trust.get('actual_violation_count')}]))
print(with_trust['assumption'])
```

1. Call the same response function twice
2. Protected unavailable → HOLD
3. Protected available → constrained injection proposal
4. Inspect the stated trust assumption

> **The correct unavailable-data result is an explicit HOLD.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

CYBER RESPONSE

Lab: connect the new estimate to a physical consequence

140–145 min · 5-minute exercise

01 Run L9

02 Check the 0.18 p.u. limit on branch 1–2

03 Inspect every other branch

04 Describe why protected information changes the action

What causes the improved physical outcome here?

The protected-state estimate and the constrained numerical control, under the stated assumptions.

> **Explain the chain from information to command to branch flow.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

The control consequence makes the failure visible

Original four-bus fixture

Information basis	Injection u	Actual flow 1–2
Stealth-corrupted	0 p.u.	0.24000 p.u.
Assumed protected	0.12169 p.u.	0.17046 p.u.

Why is the actual-flow column labeled evaluation-only?

Corrupted-state control: u = 0

Its actual branch 1–2 flow: 0.24000 p.u.

Protected-state control: u = 0.12169 p.u.

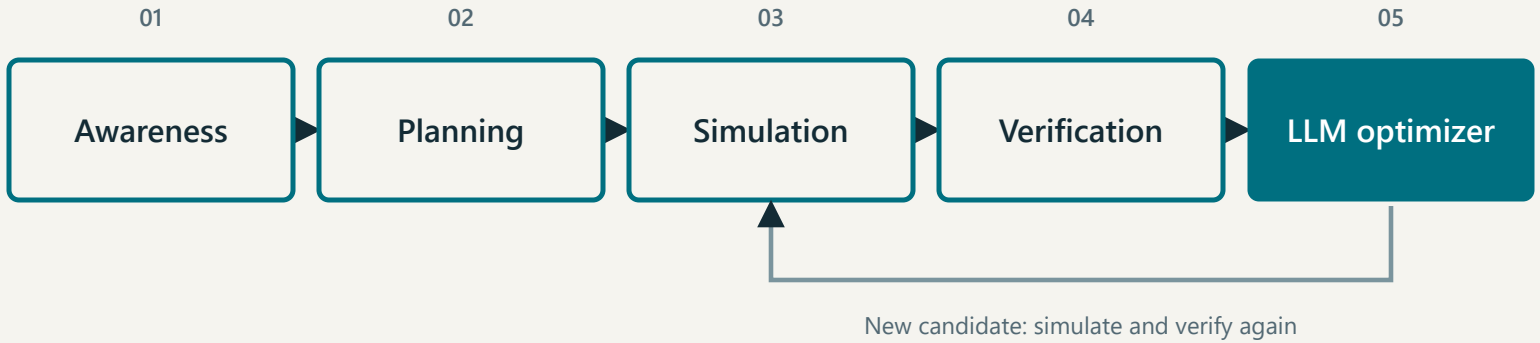
Its actual branch 1–2 flow: 0.17046 p.u.

> **A plausible corrupted estimate can leave a real overload untreated.**

Executed tutorial · Classroom Labs Schweppe & Wildes · state estimation (1970) Liu, Ning & Reiter · false-data injection (2009)

Preserve uncertainty through all five stages

Stages 1–5 · Cyber workflow



Awareness: provenance, rank and residual evidence

Planning: WLS estimate and constrained QP action

Simulation and verification: predicted flows and binding

LLM optimizer: permitted refinement, public rationale or escalation

The architecture should carry trust assumptions to the operator.

Executed tutorial · Classroom Labs Recorded experiment · five-stage workflow Schweppe & Wildes · state estimation (1970)

Recorded Llama refinement does not improve the final control

Archived actual cyber workflow

Archived cyber route	Final u	Recorded total
Fixed supervisor	0.12169 p.u.	0.0038 s
Llama supervisor	0.12169 p.u.	5.203 s

Actual planning reply: “Requesting new candidates based on the given tools and objective.”

Inspect the returned candidate and fresh numerical evidence before accepting the explanation.

> **A numerical baseline remains the reference for claimed agentic benefits.**

Recorded experiment · five-stage workflow Executed tutorial · Classroom Labs

Three cases, three different failure boundaries

147–149 min · Synthesis

Case	Failure boundary	Evidence response
Voltage	Omitted constraints	Broader-model audit
Restoration	Stale recommendation	Fresh case-bound solve
Cyber	Consistent wrong state	Justified trusted source / HOLD

Which stage would you strengthen first for each failure boundary?

Voltage: reduced scope can omit binding limits

Restoration: changed conditions invalidate an old plan

Cyber: consistent data can describe the wrong state

Common response: justified tools, fresh checks and operator review

> **A common workflow manages evidence; each use case still needs its own physics.**

Executed tutorial · Classroom Labs Recorded experiment · LLM response selection Recorded experiment · five-stage workflow

EVALUATION

What did the framework actually improve?

149–150 min · evidence before claims

Which claim is supported by a low cost and a fluent explanation?

Only the measured cost is established; feasibility and explanation support require separate evidence.

Study completed?

Constraints passed?

Claims supported?

Operator decision improved?

> **Measure the workflow and the electrical outcome separately.**

PowerAgentBench-SS (2026), §§II–III Executed tutorial: Classroom_Labs, L0–L12

EVALUATION

Compare methods on the same task

150–151 min · controlled comparison

$$\hat{p}_{\mathrm{pass}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left[g(x_i, u_i) \leq 0\right]$$

N number of evaluated cases g vector of checked constraint violations $1[\cdot]$ one when every listed constraint passes i case index x_i evaluated state in case i u_i candidate control in case i

Same case and admissible actions

Same candidate trial budget

Same final verifier

Separate failures from feasible costs

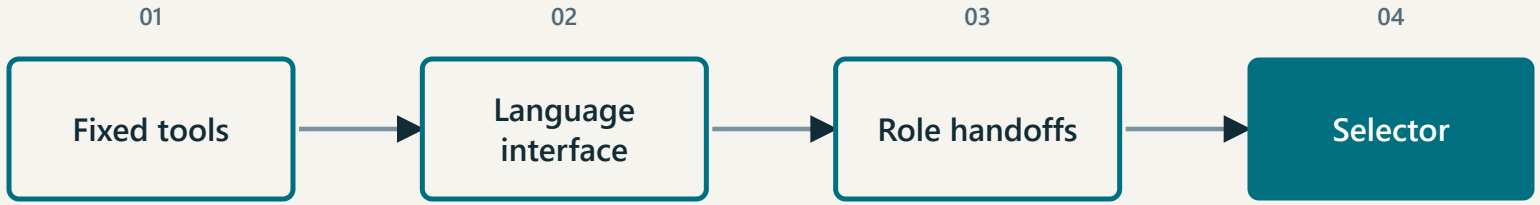
> **Give every method the same physical constraints and verification budget.**

PowerAgentBench-SS (2026), §§II–III Recorded voltage pilot: E1–E8

EVALUATION

Ablate one mechanism at a time

151–152 min · experiment design



Fixed numerical workflow

Add language interpretation

Add role-specific handoffs

Add vote or learned selection

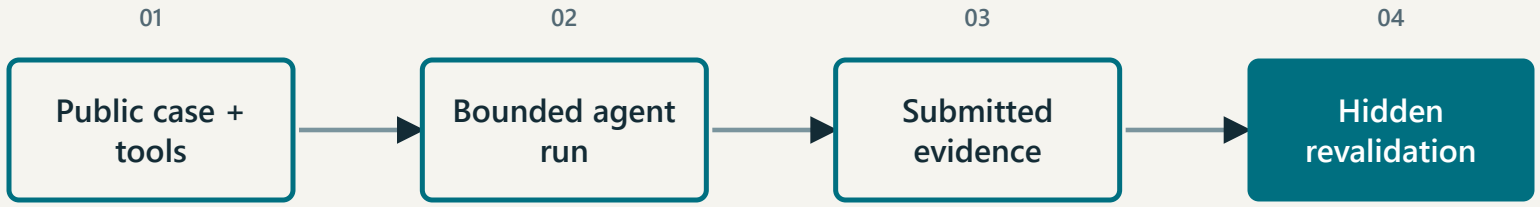
> **A useful ablation keeps the numerical tools fixed.**

PowerAgentBench-SS (2026), §§II–III Executed LangGraph teaching lab: G0–G5 Recorded voltage pilot: E1–E8

EVALUATION

Frontier: evidence-backed evaluation

152–153 min · PowerAgentBench-SS



✓ Public tools and fixed budgets

✓ Hidden physical revalidation

✓ Evidence-supported submissions

Scope: DC thermal pilot

> **The final report should survive independent recomputation.**

PowerAgentBench-SS (2026), §§II–III

EVALUATION

Frontier: request uncertainty studies

153–154 min · CONDUCTOR

01 ✓ Numerical engines remain explicit

02 ✓ Uncertainty-aware study selection

03 ✓ Corrective-dispatch tools

04 Measurement replay is not forecast validation

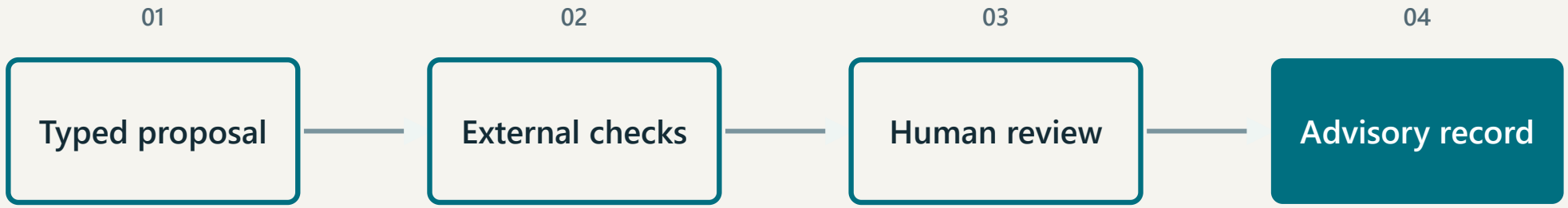
> **The orchestrator can request a stronger study when uncertainty matters.**

CONDUCTOR (2026), uncertainty-aware orchestration Executed tutorial: Classroom_Labs, L0–L12

EVALUATION

Frontier: enforce authority in software

154–155 min · control-room governance



✓ Whitelisted tools

✓ Explicit step budgets

✓ Typed numerical facts

✓ Review before side effects

> **Approval rules belong in the runtime, not only in the prompt.**

Governance-aware TSO digital twin (2026) Executed tutorial: Classroom_Labs, L0–L12

L11: one optional local Llama call

155–156 min · prepare Ollama before class

Classroom_Labs.ipynb · L11 · lines 6–8

```
6     client = ProposalLlama(journal, model='llama3.1:8b')
7     live_reply = client.ask('Optimization Agent', pool['context']
  ['public_context'],
8                             list(pool['context']['catalog']))
```

1

Default: no new model call

2

Enable RUN_LOCAL_LLAMA in L0 for a new call

3

The saved response is from original test_01

Recorded Llama reason: “has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost.”

The 17 October validation chose RELAX_Q_10_PERCENT. Test it before accepting it.

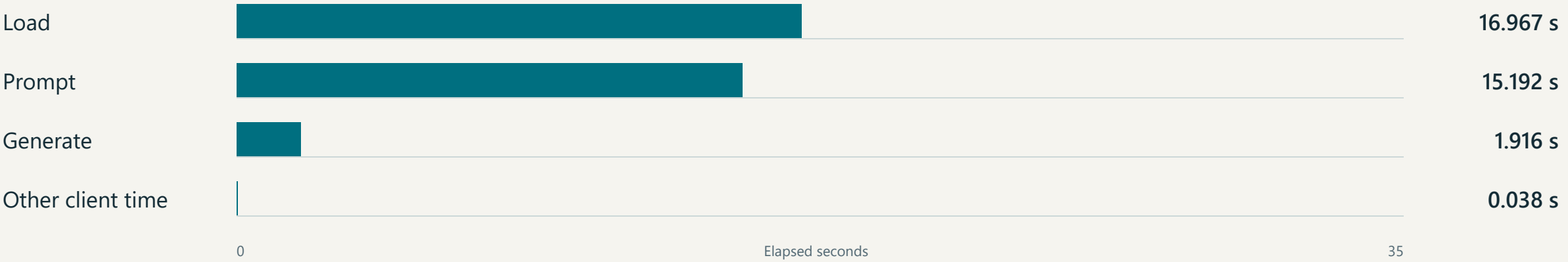
> **An actual model response proposes a trial; the numerical verifier still decides admissibility.**

Ollama chat API Classroom L11 and saved actual validation

EVALUATION

Where did one Llama call spend its time?

156–158 min · actual local call, 17 October 2026



Generation: 1.916 s

Client elapsed: 34.113 s

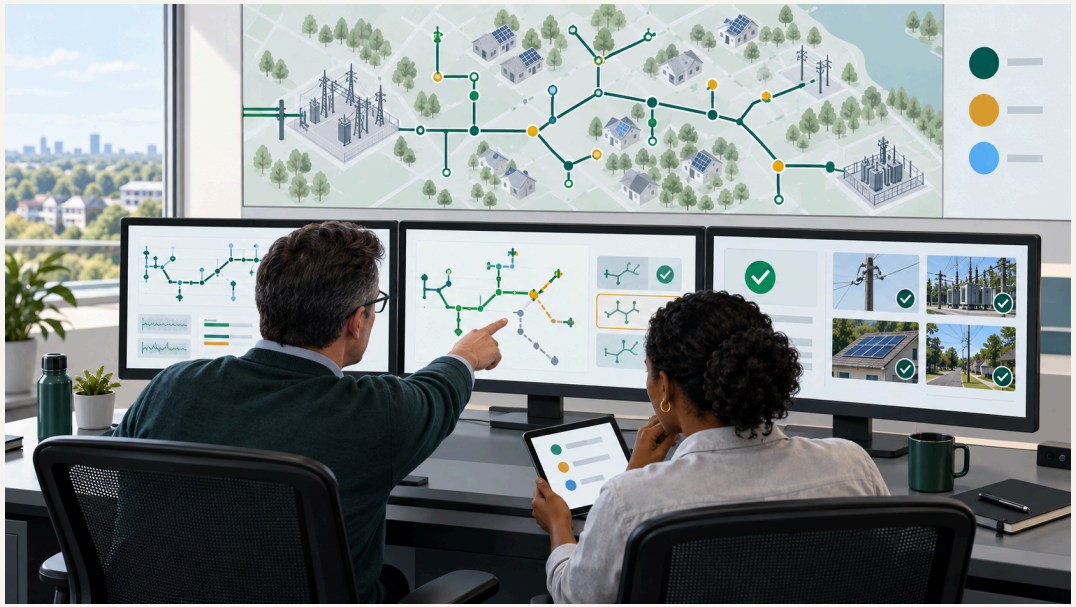
Report loading, prompt processing and generation separately.

Ollama chat API: timing fields Executed tutorial: Classroom_Labs, L0–L12

CONTROL ROOM

Place the assistant beside the control-room tools

158–159 min · deployment concept



Conceptual AI-generated illustration, not a utility facility or deployed system.

- Existing operational data interfaces
- Sandboxed numerical studies
- Evidence-backed recommendation
- Authorized operator procedure

The tutorial is an advisory sandbox with an explicit human boundary.

Governance-aware TSO digital twin (2026) Executed tutorial: Classroom_Labs, L0–L12

The operator sees scope before readback

159–160 min · saved L12 operator packets

Packet field	Full MV/LV request	Explicit MV teaching request
Verified scope	MV phases only	MV phases only
Scope matches	No	Yes
State	HOLD	READY_FOR_READBACK
Acknowledged	False	True
Equipment dispatch	False	False

Both packets refer to b67cff9c8eb444c2. What changes the result?

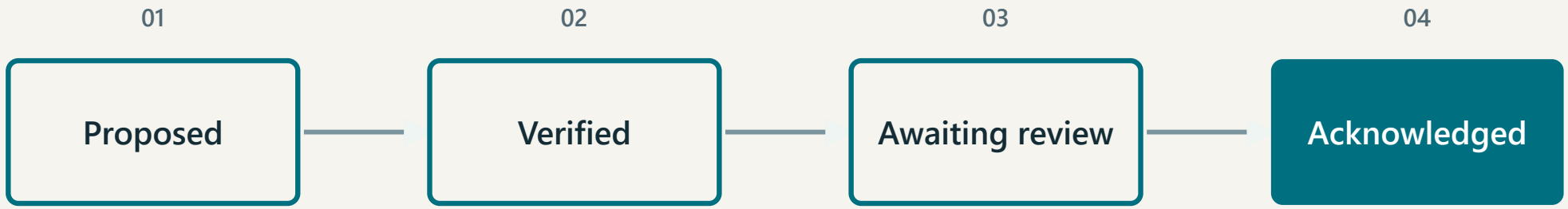
The requested scope must match the verified scope; the recorded readback remains advisory.

> **Lead with action, scope, margins and unresolved issues.**

CONTROL ROOM

Acknowledgment is a guarded transition

160–161 min · state-machine view



Proposed → checked

Checked → awaiting review

Matching readback → acknowledged

Scope or identity failure → HOLD

> **Only a matching, eligible packet can be acknowledged.**

Executed tutorial: Classroom_Labs, L0–L12 Executed LangGraph teaching lab: G0–G5 LangGraph: interrupts and resume

Code: bind review to the candidate

161–162 min · Classroom_Labs L12

Classroom_Labs.ipynb · L12 · lines 9–10

```
 9 wrong_id = lab.acknowledge(mv_scope, 'wrong-id', 'Student operator')
10 readback = lab.acknowledge(mv_scope, mv_scope['recommendation_id'],
   'Student operator')
```

1
First request uses a wrong ID

2
Second reads back the packet ID

3
The operator label is a classroom placeholder

> **A wrong candidate ID cannot acknowledge the packet.**

Executed tutorial: Classroom_Labs, L0–L12

CONTROL ROOM

Hands-on: read back, reject, inspect

162–166 min · four-minute exercise

01 Run L12 after its earlier cells

02 Find the full-scope HOLD

03 Inspect the wrong-ID rejection

04 Confirm no equipment dispatch

Which of the three outcomes is permission to dispatch equipment?

None. Even the acknowledged packet is explicitly advisory.

> **Use L12 to distinguish scope, identity and acknowledgment.**

Executed tutorial: Classroom_Labs, L0–L12

The checks distinguish three outcomes

166–167 min · lab debrief

Classroom_Labs.ipynb · L12 · lines 13–15

```
13 assert full_scope['state'] == 'HOLD' and not full_scope['acknowledged']
14 assert wrong_id['acknowledgment_result'] == 'REJECTED'
15 assert readback['acknowledged'] and not readback['equipment_dispatch']
```

1. Full MV/LV requirement: HOLD
2. Wrong identifier: REJECTED
3. Matching MV readback: acknowledged
4. equipment_dispatch: false

> **Scope mismatch holds; wrong identity rejects; valid readback remains advisory.**

Executed tutorial: Classroom_Labs, L0–L12

CONTROL ROOM

A changed request needs new evidence

167–168 min · stale-evidence challenge

Can the old acknowledgment survive a changed control?

The changed candidate needs new checks and a new matching review record.

Change a limit or candidate

Old binding no longer matches

Recompute the affected study

Review the new packet

> **A hash links content; it does not establish physical truth.**

Executed tutorial: Classroom_Labs, L0–L12

CONTROL ROOM

Capstone: approve review, hold or request a study?

168–171 min · group decision

For your card, identify one missing fact and the next tool or human step.

A: broader-model repair. B: authorized isolation evidence. C: justified independent measurements or escalation.

A: MV PASS; full-feeder AC FAIL

B: restoration PASS; isolation record absent

C: no alarm; protected measurements absent

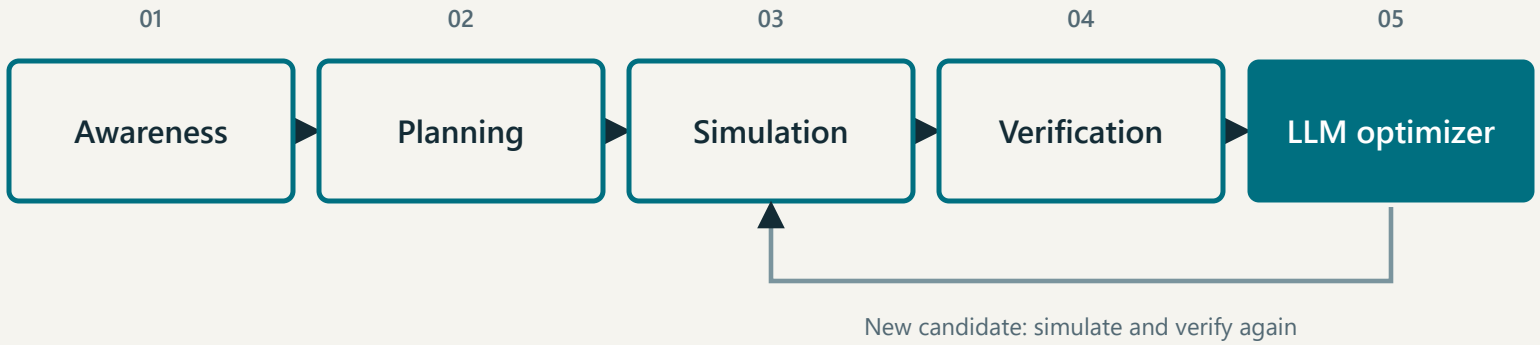
> **Name the missing evidence and the next responsible step.**

Executed tutorial: Classroom_Labs, L0–L12 Liu, Ning & Reiter, false-data injection, CCS 2009

SYNTHESIS

Five stages, three cases, one evidence discipline

171–173 min · transferable structure



Awareness: establish the study context

Planning: generate admissible candidates

Simulation + verification: test and accept

Optimizer: refine; operator: review

> **Each stage contributes a method and a checkable output.**

Executed tutorial: Classroom_Labs, L0–L12 Recorded voltage pilot: E1–E8

SYNTHESIS

Choose the simplest sufficient workflow

173–174 min · architecture selection

01 Repeated calculation → fixed workflow

02 Varied requests → typed LLM interface

03 Independent studies → specialist coordination

04 Consequential recommendation → enforced review

> **Add an agent only when it addresses a measured task need.**

Kim et al., scaling agent systems (2026) Executed LangGraph teaching lab: G0–G5

SYNTHESIS

A reproducible result carries its context

174–175 min · scientific reporting

01 Case, model scope and data revision

02 Solver, policy and LLM versions

03 Seeds, budgets and measured durations

04 Fresh, archived or optional-live label

> **Archive the inputs, methods, outputs and evidence mode together.**

Executed tutorial: Classroom_Labs, L0–L12 Recorded voltage pilot: E1–E8 Ollama chat API: timing fields

SYNTHESIS

Design your next experiment

175–176 min · research extension

01 Hypothesis: name the expected benefit

02 Control: retain a strong fixed baseline

03 Measure: outcome, failures and latency

04 Boundary: state the model and task scope

Write one testable benefit and one result that would refute it.

Example: fewer unsupported reports at equal tool budget; equal or higher unsupported-report rate refutes the claimed benefit.

> **Change one factor and state what would falsify your claim.**

PowerAgentBench-SS (2026), §§II–III Kim et al., scaling agent systems (2026)

SYNTHESIS

Exit ticket: explain, run, interpret

176–179 min · individual reflection

Which checkpoint would you show a colleague to explain the framework?

Choose a checkpoint and support the explanation with its exact inputs, output and stated limitation.

Explain a DAG versus a bounded cycle

Name a model-based and model-free method

Interpret one failed candidate correctly

Identify the operator acknowledgment boundary

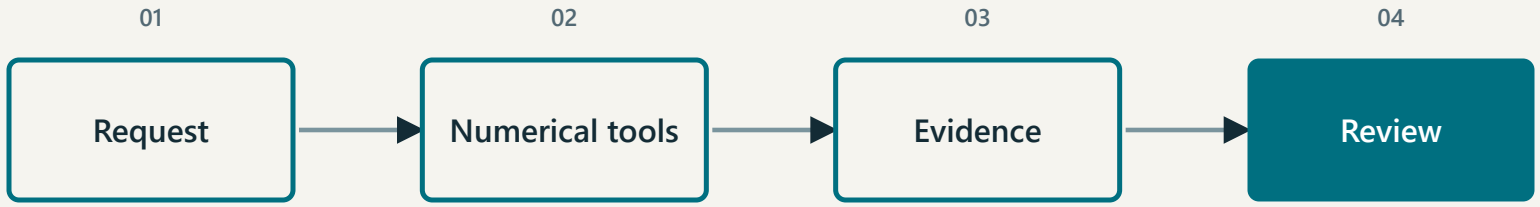
> **Connect one concept, one cell and one observed result.**

Executed LangGraph teaching lab: G0–G5 Executed tutorial: Classroom_Labs, L0–L12

SYNTHESIS

From request to a reviewable recommendation

179–180 min · take-home principle



Language expresses the task

Numerical methods produce candidates

Checks determine what is supported

People retain the review boundary

> **Agentic assistance earns trust through useful tools, explicit evidence and accountable decisions.**

Executed tutorial: Classroom_Labs, L0–L12 Executed LangGraph teaching lab: G0–G5 Recorded voltage pilot: E1–E8